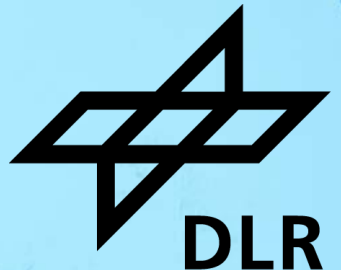


INTRODUCTION TO DEEP LEARNING

PART I – INTRODUCTION AND BASICS

Auliya Fitri, Sheetal Sahoo, Sreerag Naveenachandran

**Machine Learning Group
Institute of Data Science**



Sreerag Vadakkemeppully Naveenachandran

German Aerospace Center – Institute of Data Science
Data Analysis and Intelligence
Machine Learning Group

Research Interests:

- Machine learning for engineering systems
- Anomaly detection in time series
- Diffusion models

Contact:

Phone: +49 3641 30960 206

Mail: Sreerag.Naveenachandran@dlr.de



Auliya Fitri

German Aerospace Center – Institute of Data Science

Data Analysis and Intelligence

Machine Learning Group

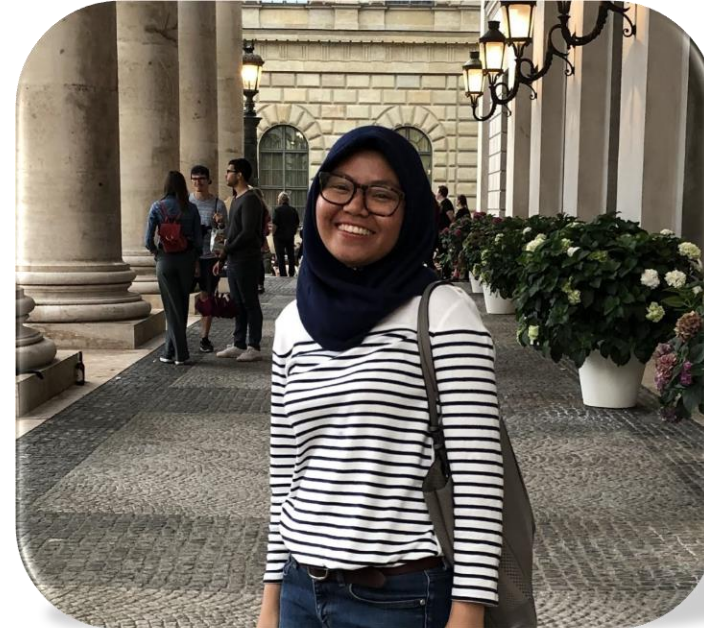
Research Interests:

- Machine Learning
- Explainable Artificial Intelligence
- Uncertainties in Neural Networks

Contact:

Phone: +49 3641 30960 165

Mail: aulya.fitri@dlr.de



Sheetal Sahoo

German Aerospace Center – Institute of Data Science
Data Analysis and Intelligence
Machine Learning Group

Research Interests:

- Anomaly Detection in Time series
- Privacy Preserving Machine Learning

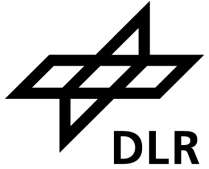
Contact:

Phone: +49 3641 30960 204

Mail: sheetal.sahoo@dlr.de



Schedule



Date	Time	Activity
28.05.2026 Day 1	09:00 - 10:00	Introduction and basics
	10:00 - 10:30	Hands-on I
	10:30 - 10:45	Coffee break
	10:45 - 11:15	Hands-on II
	11:15 - 12:00	Hands-on III (Group Work)
	12:00 - 13:00	Lunch break
	13:00 - 14:00	Advanced concept and Convolutional Neural Network
	14:00 - 14:45	Hands-on IV
	14:45 - 15:15	Recap Day 1
29.05.2026 Day 2	09:00 - 10:00	Deep Generative Models
	10:00 - 10:45	Hands-on III
	10:45 - 11:15	Latest applications, code and knowledge sources
	11:15 - 11:30	Closing

I. Introduction and basics

- Application examples
- Machine learning background
- Neural network concepts
- Training procedure

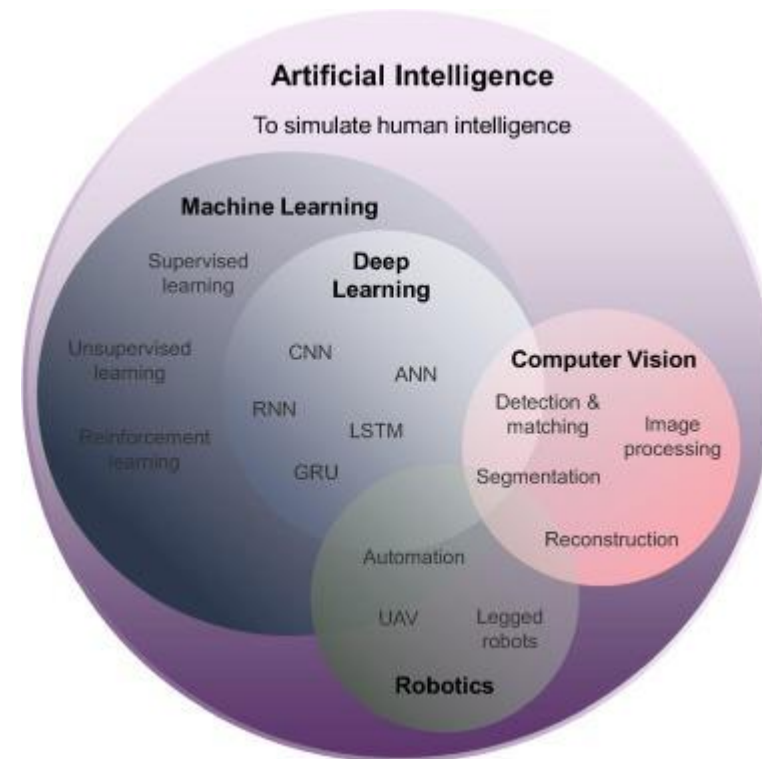
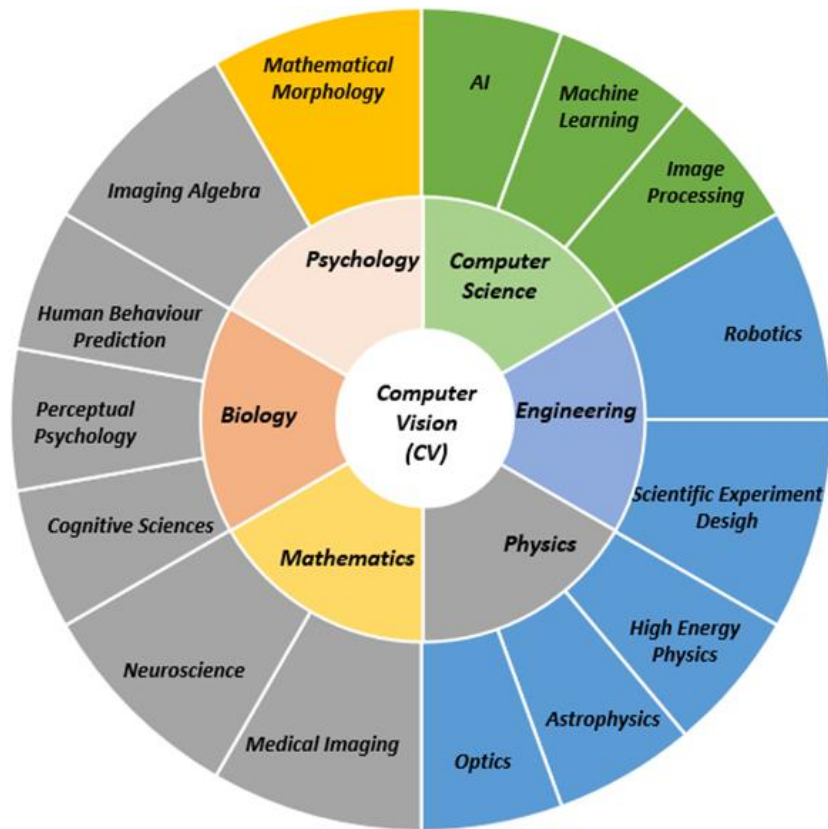
Inspired by lectures from Paris Saclay and MIT; images taken from these, if not noted otherwise

APPLICATION EXAMPLES

Application Examples

Computer Vision

Computer Vision is a interdisciplinary field with strong relations to AI and DL



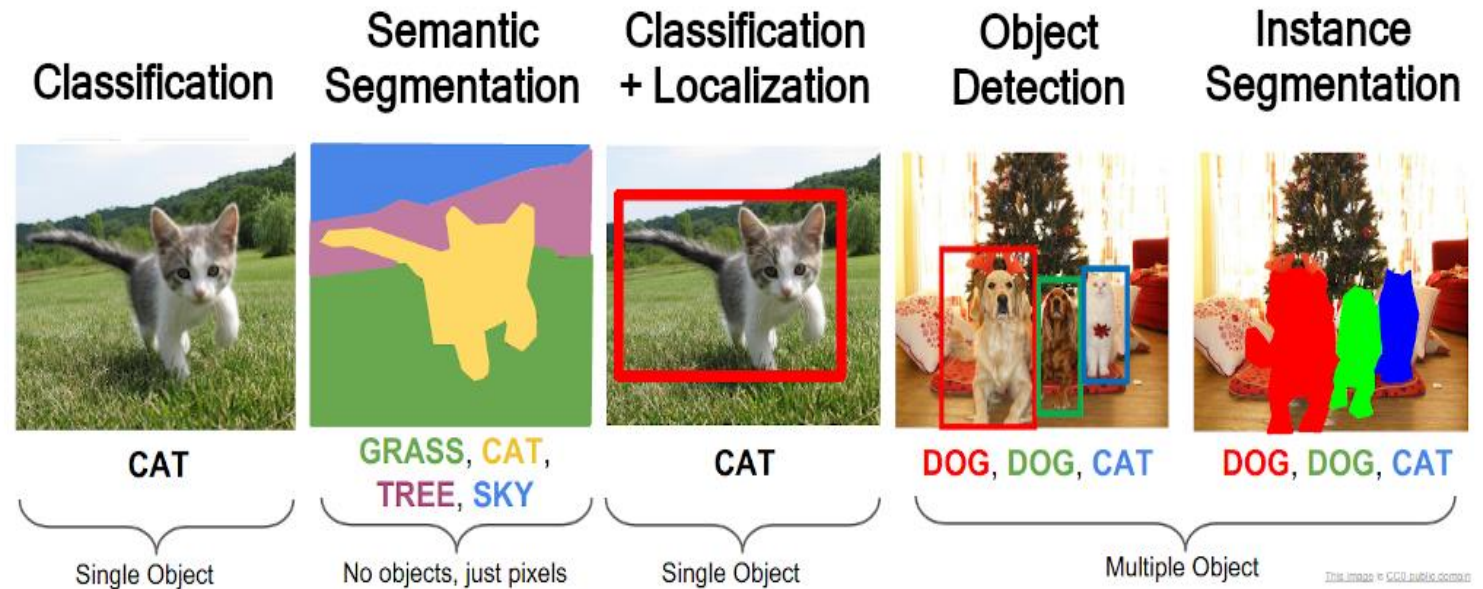
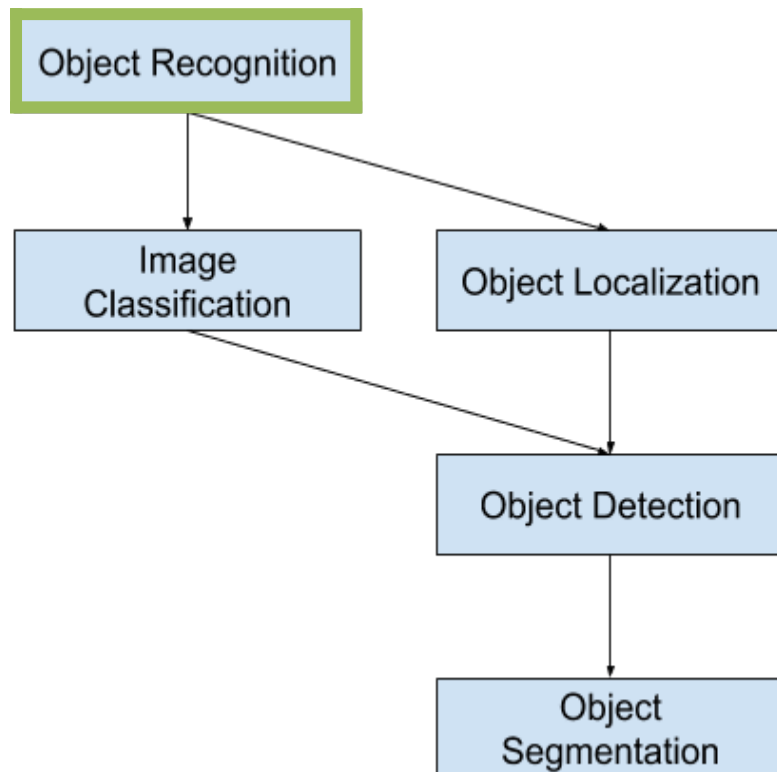
[Computer Vision and Deep Learning – SevenShineStudios \(wordpress.com\)](https://www.sevenshinestudios.com/computer-vision-and-deep-learning/)

[BIM, machine learning and computer vision techniques in underground construction: Current status and future perspectives - ScienceDirect](#)

Application Examples

Computer Vision - Object Recognition / Image Processing

“Object Recognition refers to a collection of related tasks for identifying objects in digital photographs.”



http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf

A Gentle Introduction to Object Recognition With Deep Learning (machinelearningmastery.com) (2021)

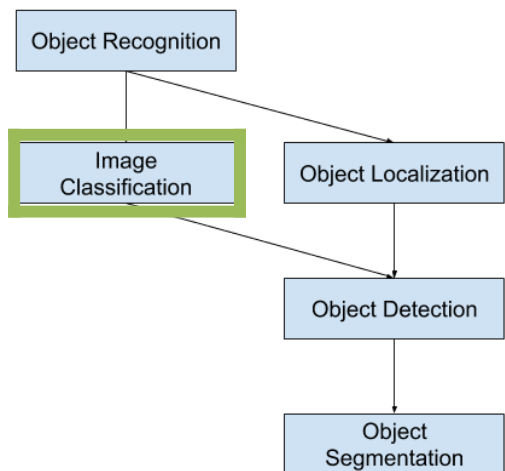
Application Examples

Computer Vision – Image Classification

“Image Classification predicts the type or class of an object in an image.”



CAT



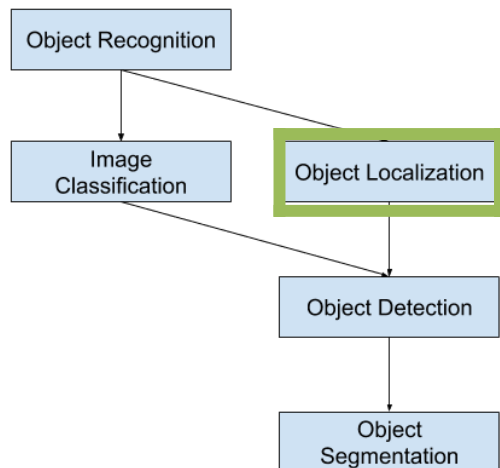
Application Examples

Computer Vision – Object Localization

“Object Localization locates the presence of objects in an image and indicate their location with a bounding box.”



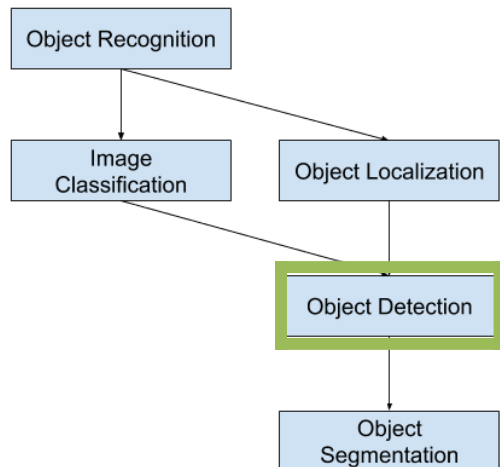
CAT



Application Examples

Computer Vision – Object Detection

“Object Detection locates the presence of objects with a bounding box and types or classes of the located objects in an image.”



Application Examples

Computer Vision – Semantic and Object Segmentation

Semantic Segmentation
highlights pixels but not objects

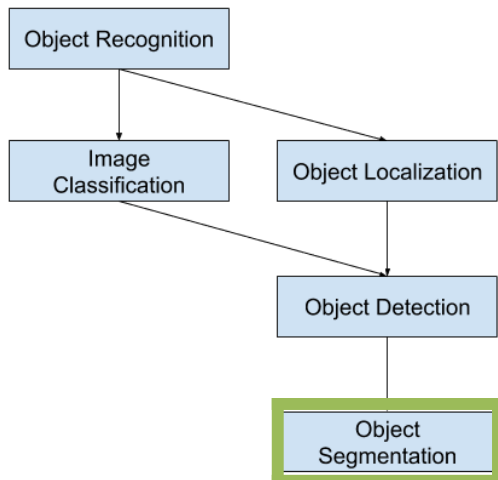


GRASS, CAT,
TREE, SKY

“Object Segmentation highlights the specific
pixels of the object instead of a coarse
bounding box.”



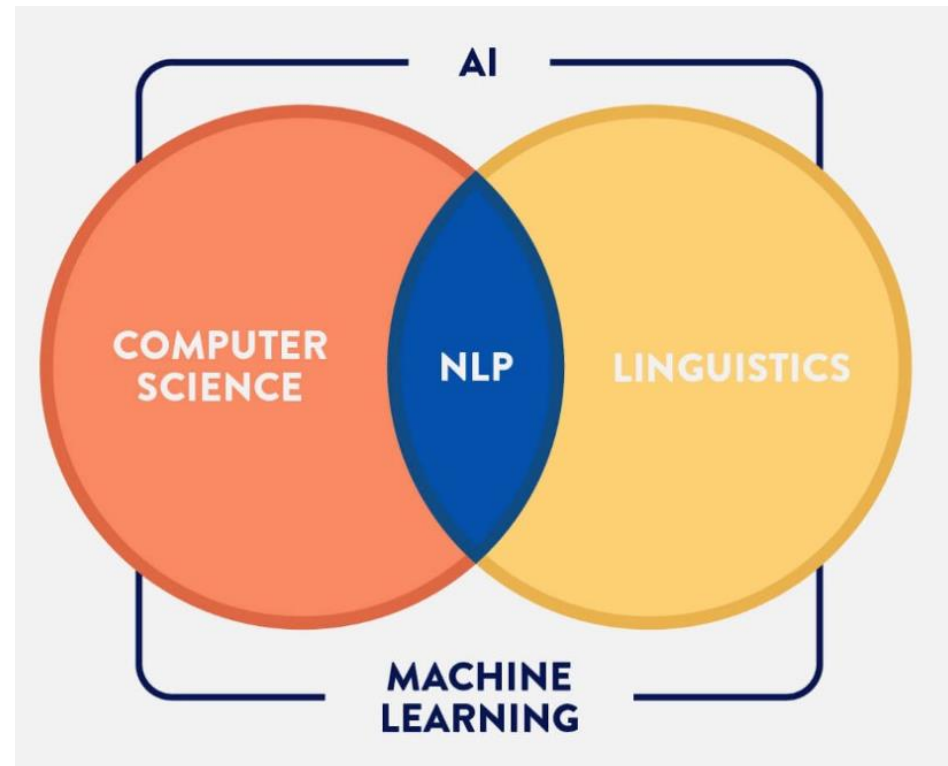
DOG, DOG, CAT



Application Examples

Natural Language Processing (NLP)

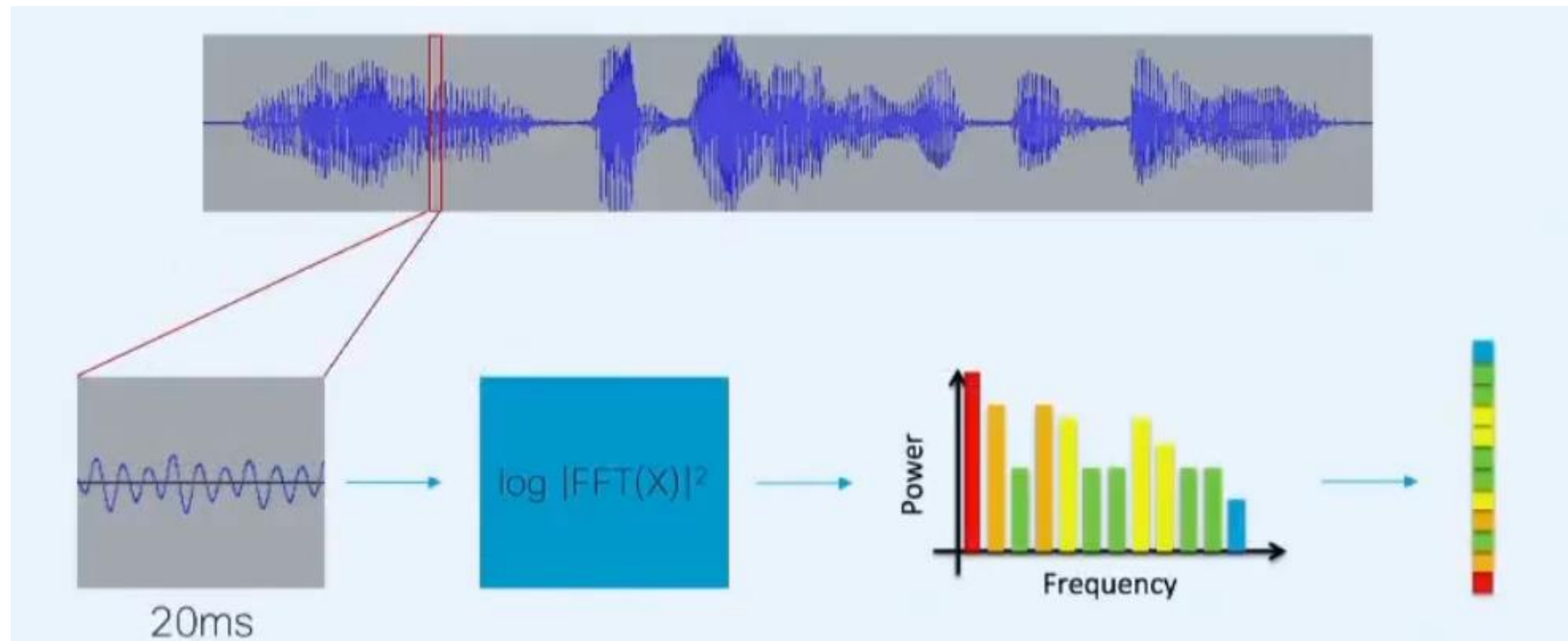
NLP is an interdisciplinary field and gives computers the ability to understand human language.



Application Examples

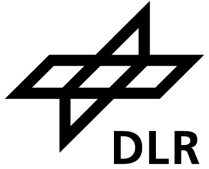
Speech Recognition

“The process of enabling a computer to identify and respond to the sounds produced in human speech.”



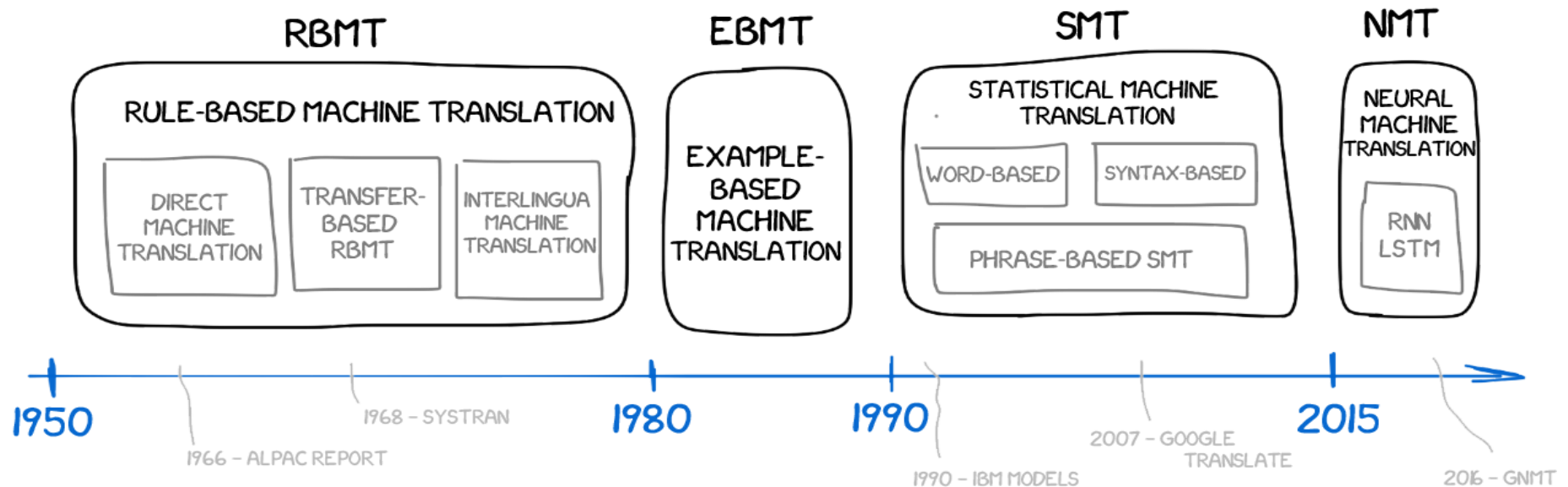
Application Examples

NLP – Language Translation / Machine Translation



“Machine translation is the process of using artificial intelligence (AI) to automatically translate content from one language (the source) to another (the target) without any human input.”

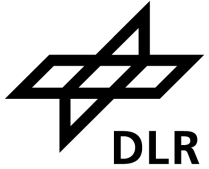
A BRIEF HISTORY OF MACHINE TRANSLATION



[Machine Translation :: From the Cold War to Deep Learning :: vas3k.com](https://vas3k.com/machine-translation-from-the-cold-war-to-deep-learning/)

Application Examples

NLP – Language Translation / Machine Translation



Google Translate

Translate Turn off instant translation

English Russian German Detect language German Russian English Translate

Take your time

14/5000

Lassen Sie sich Zeit

Suggest an edit

See also
Take your time., Take your time!, your, time

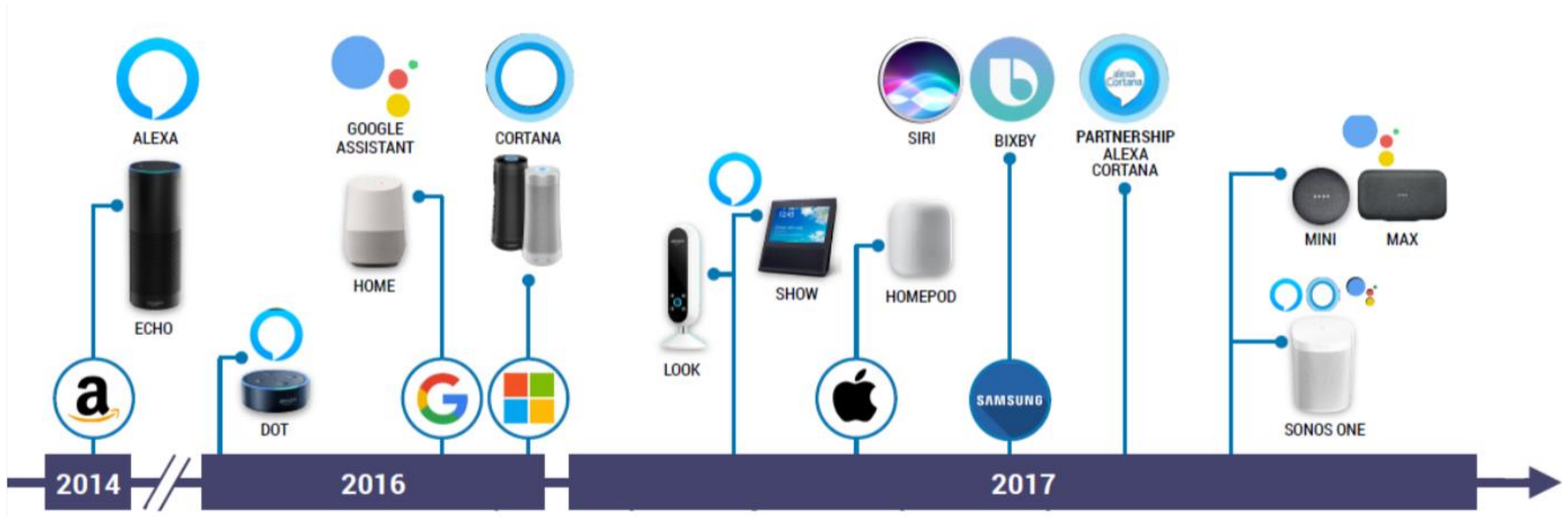
Translations of Take your time.
phrase
Nehmen Sie sich etwas Zeit. Take your time.

[Machine Translation :: From the Cold War to Deep Learning :: vas3k.com](https://vas3k.com)

Application Examples

NLP – Virtual & Voice Assistants

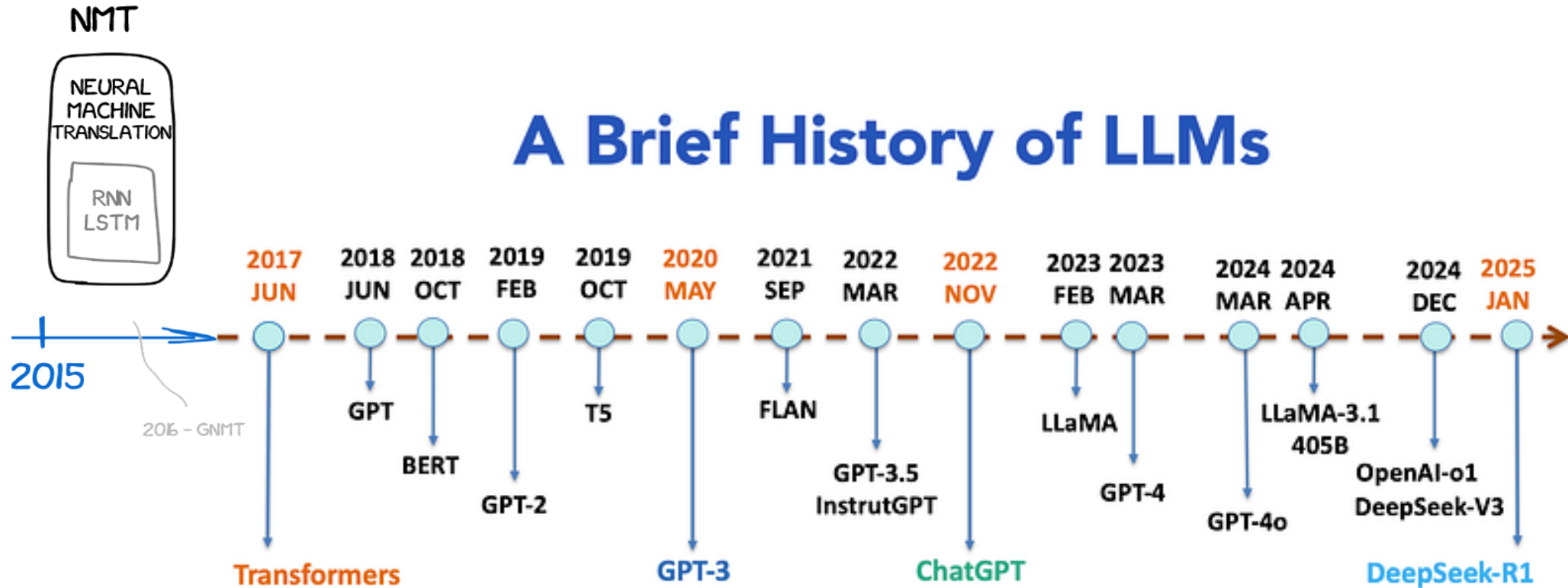
“A virtual assistant is an application that understands voice commands and completes tasks for a user”



[Haben Sie schon Ihren persönlichen Voice Assistant? \(pidas.com\)](https://www.pidas.com/)

Application Examples

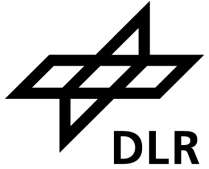
NLP – Large Language Models (LLMs)



[A brief history of LLMs \(medium.com\)](https://medium.com/@davidsonmiller/a-brief-history-of-llms-1234567890)

Application Examples

Entertainment – games & bots



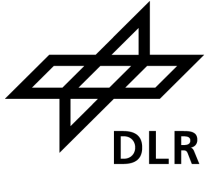
AlphaZero (deepmind) is a self-taught computer program for very high complexities. It learns with a complete game information approach solely based on game rules



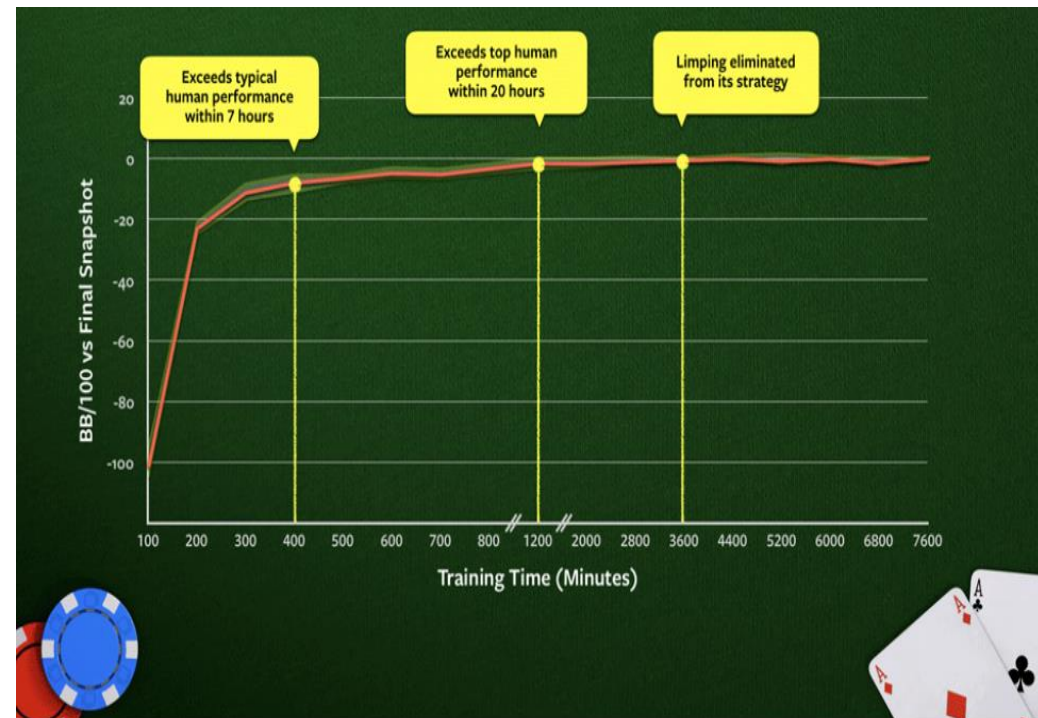
[Go \(game\) - Wikipedia](#)

Application Examples

Entertainment – games & bots



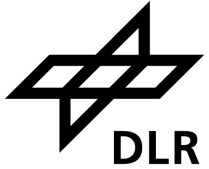
Pluribus (facebook) is an AI for multiplayer poker that beats 6 pros at once. It reaches top performances after 20h training and learns with an incomplete game information approach.



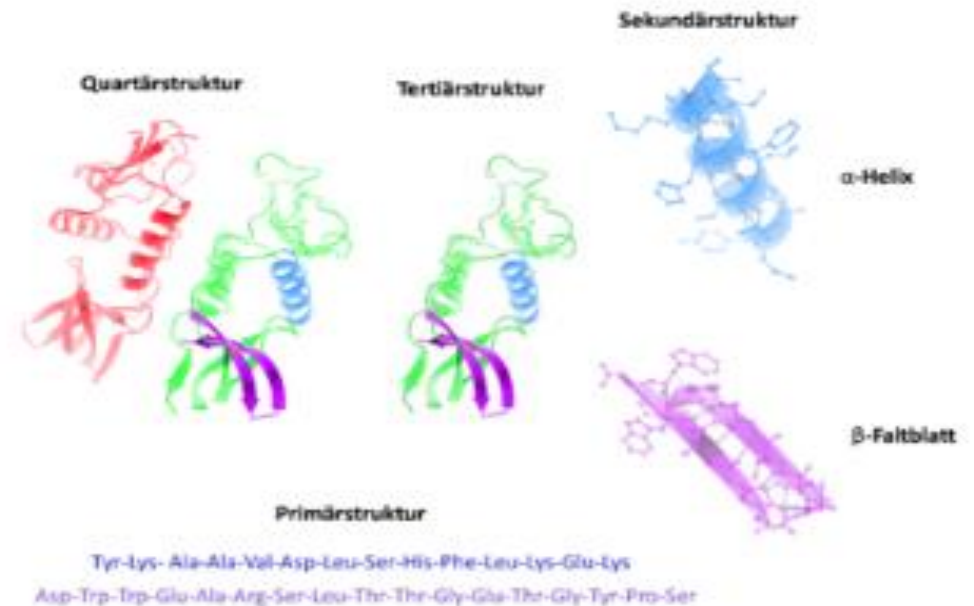
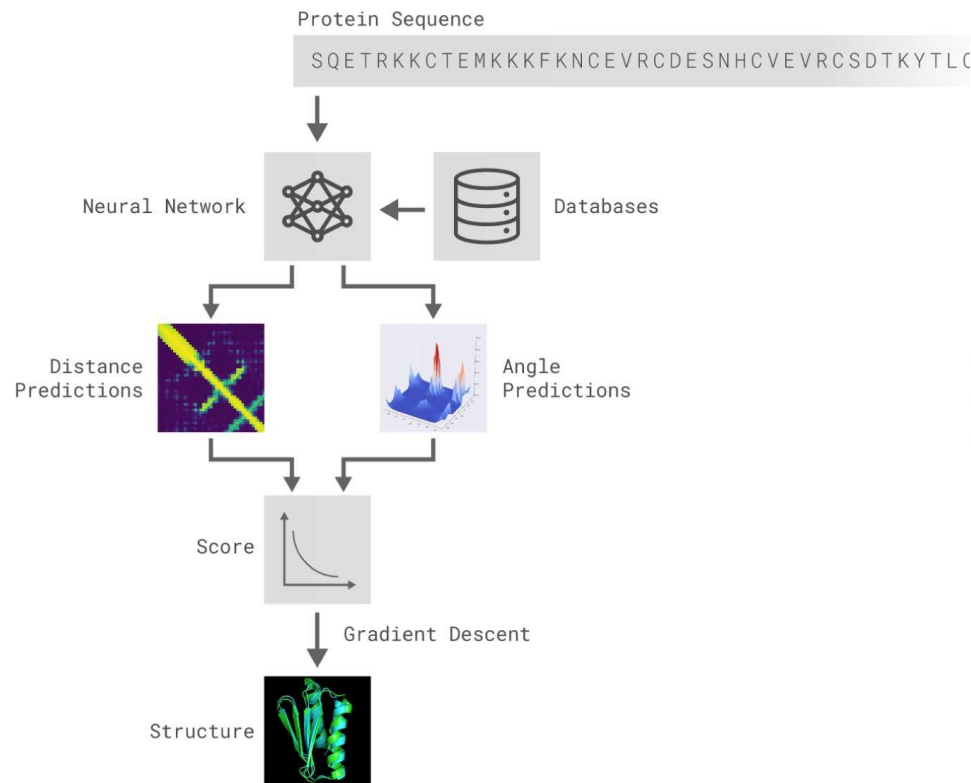
[Superhuman AI for multiplayer poker \(science.org\)](https://www.sciencemag.org/news/2017/05/superhuman-ai-for-multiplayer-poker)
[Facebook, Carnegie Mellon build first AI that beats pros in 6-player poker](https://www.facebook.com/PluribusAI/)

Application Examples

AlphaFold by DeepMind (Winner of chemistry Nobel Prize 2024)



AlphaFold predicts protein structure based on the amino acid sequence of the protein.



[AlphaFold – Wikipedia](#)

MACHINE LEARNING BACKGROUND

Machine learning background

ARTIFICIAL INTELLIGENCE

Any technique that enables computers to mimic human behavior



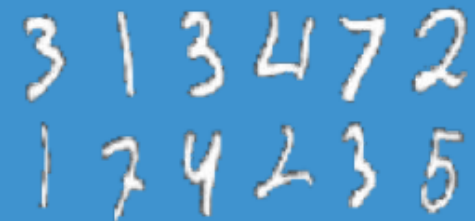
MACHINE LEARNING

Ability to learn without explicitly being programmed



DEEP LEARNING

Extract patterns from data using neural networks



Machine learning background

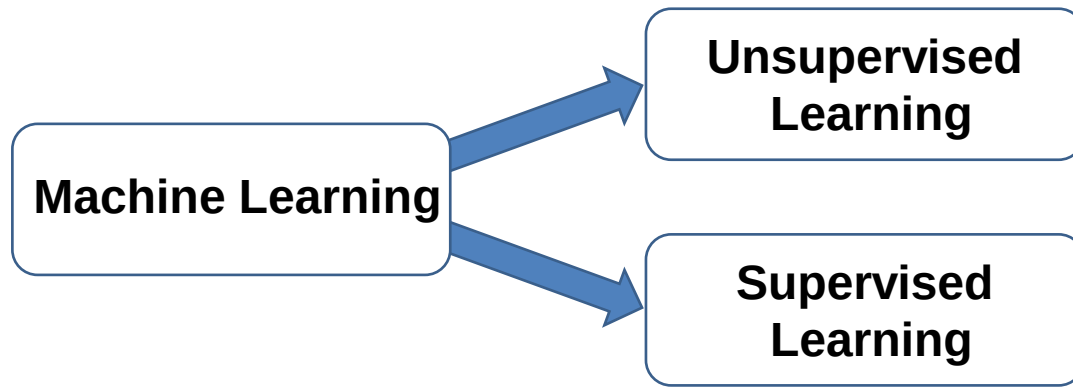
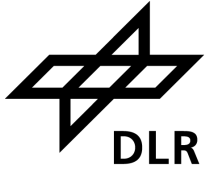
Machine Learning Technics



Machine Learning

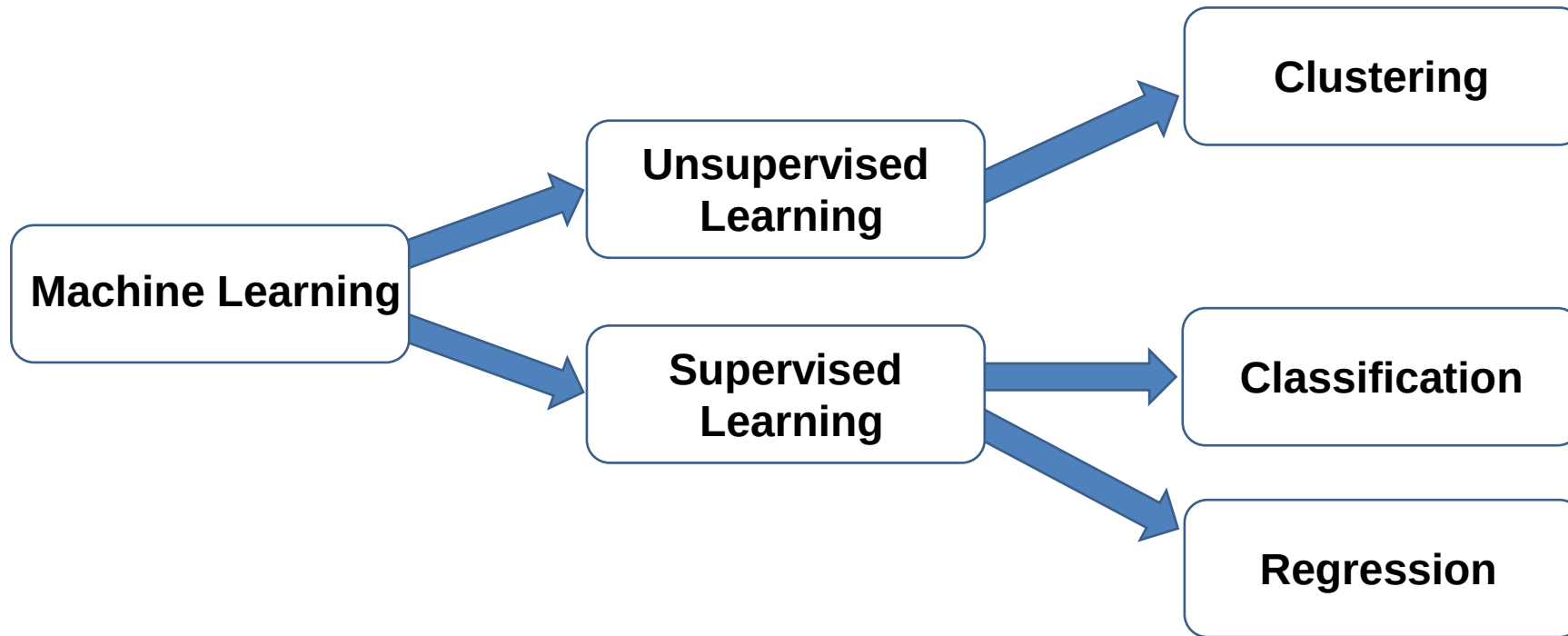
Machine learning background

Machine Learning Technics



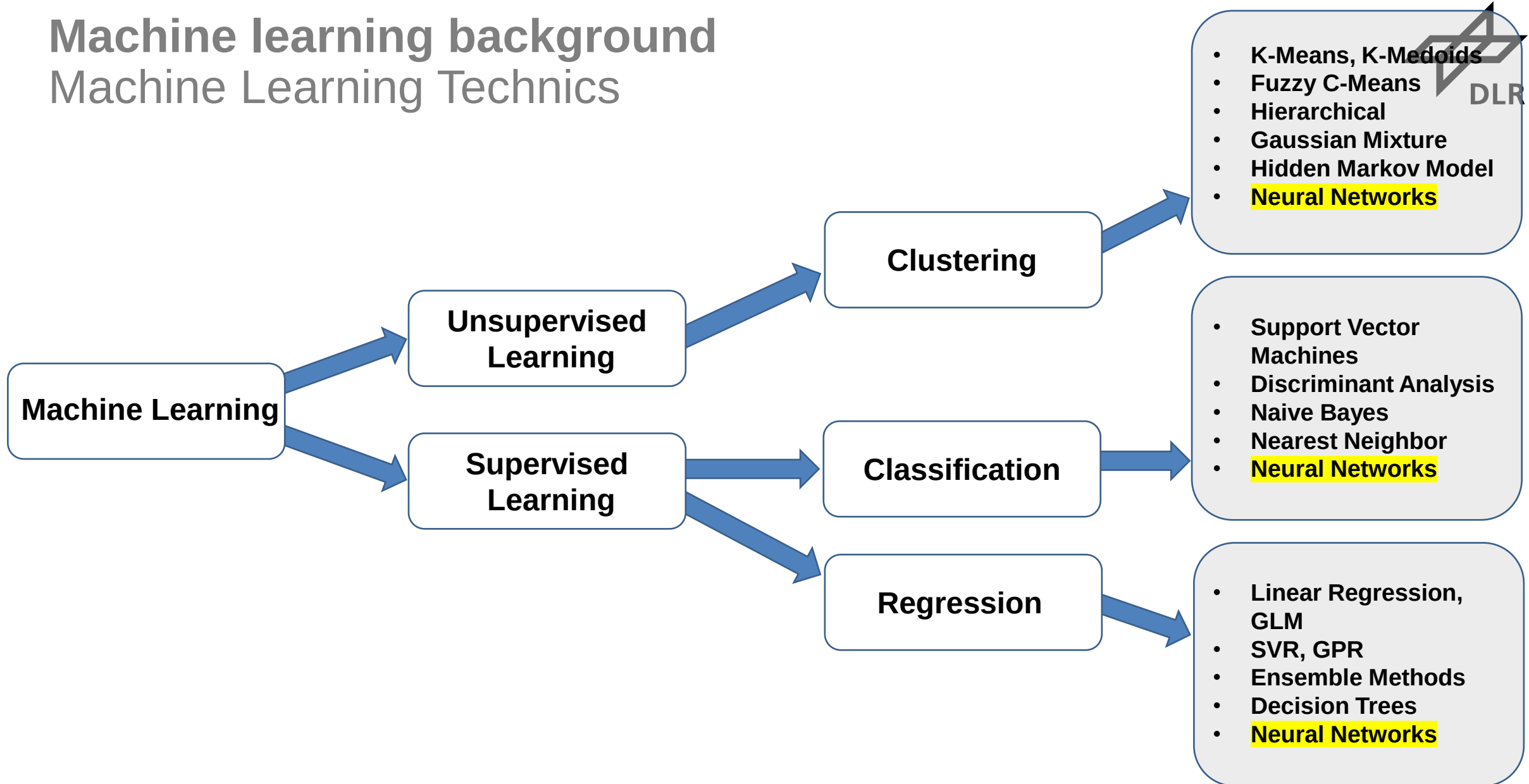
Machine learning background

Machine Learning Technics



Machine learning background

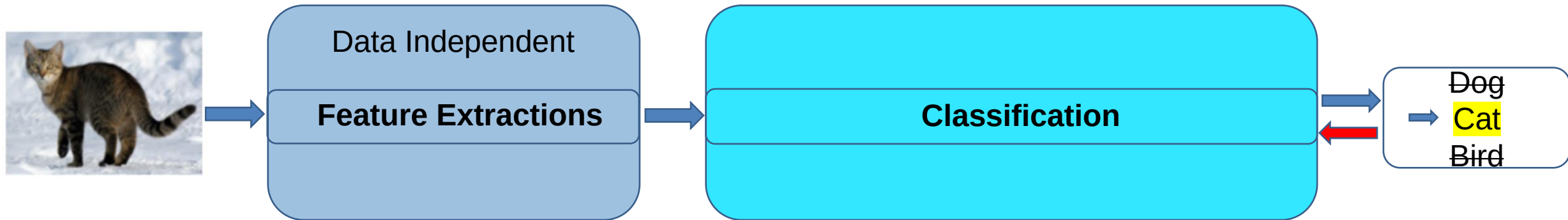
Machine Learning Technics



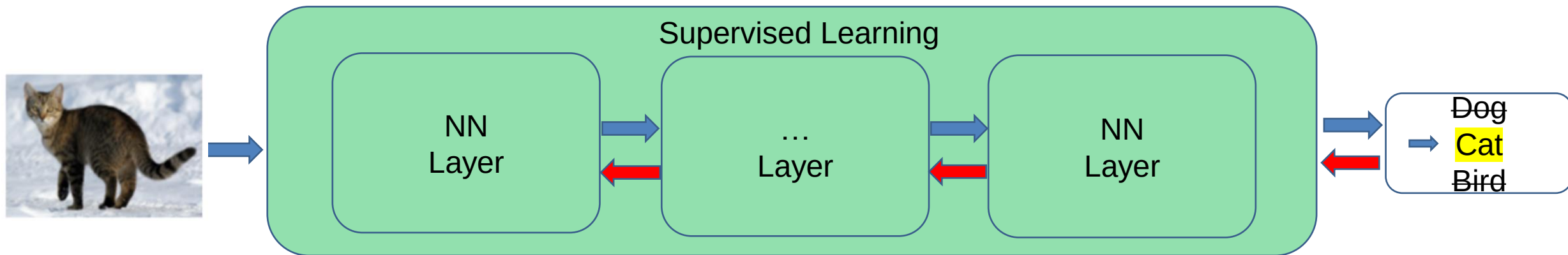
Machine learning background



“Traditional” ML System

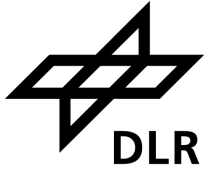


Deep Learning System



Machine learning background

Deep learning system - Why deep learning now?



1. Big Data

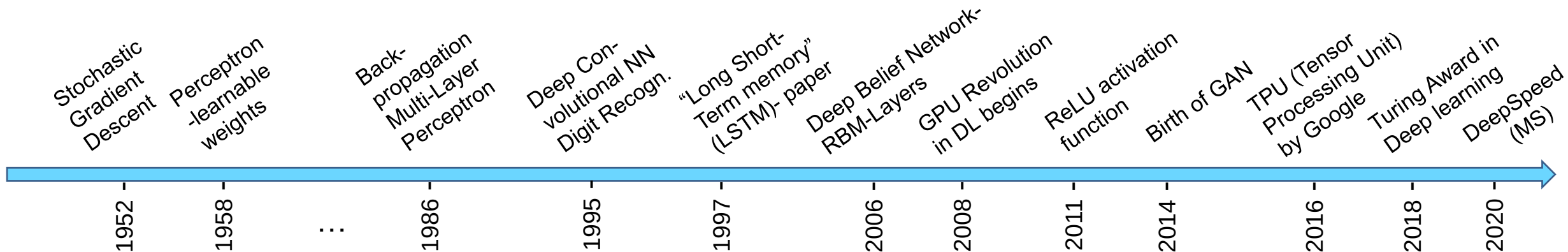
- 
- +DL superiority with increasing big data sets
 - +Collection & storage
 - +relevant for multiple disciplines
- 
- +decreasing error rate

2. Hardware

- +Parallelization (GPU/TPU) 
- +Cloud-based DL systems (e.g. AWS, MS-Azure) 
- +reduce computing Power and memory use

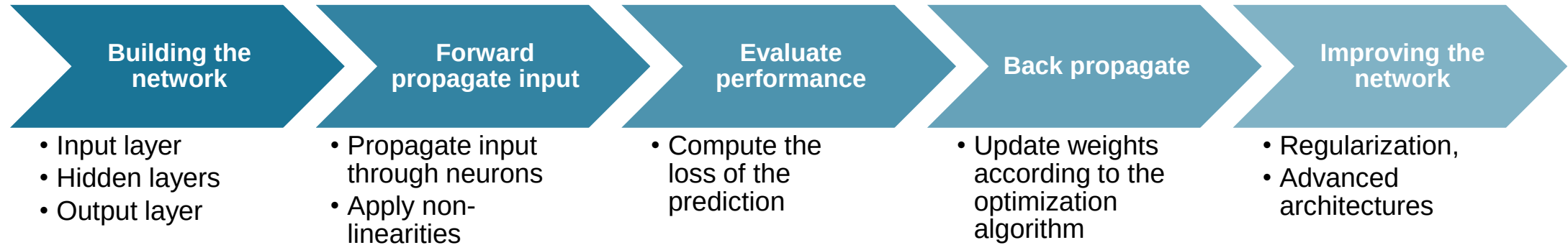
3. Software

- +improved techniques from growing research
- +optimization SW&HW
- +new models and toolboxes



NEURAL NETWORK CONCEPTS

Neural network concepts

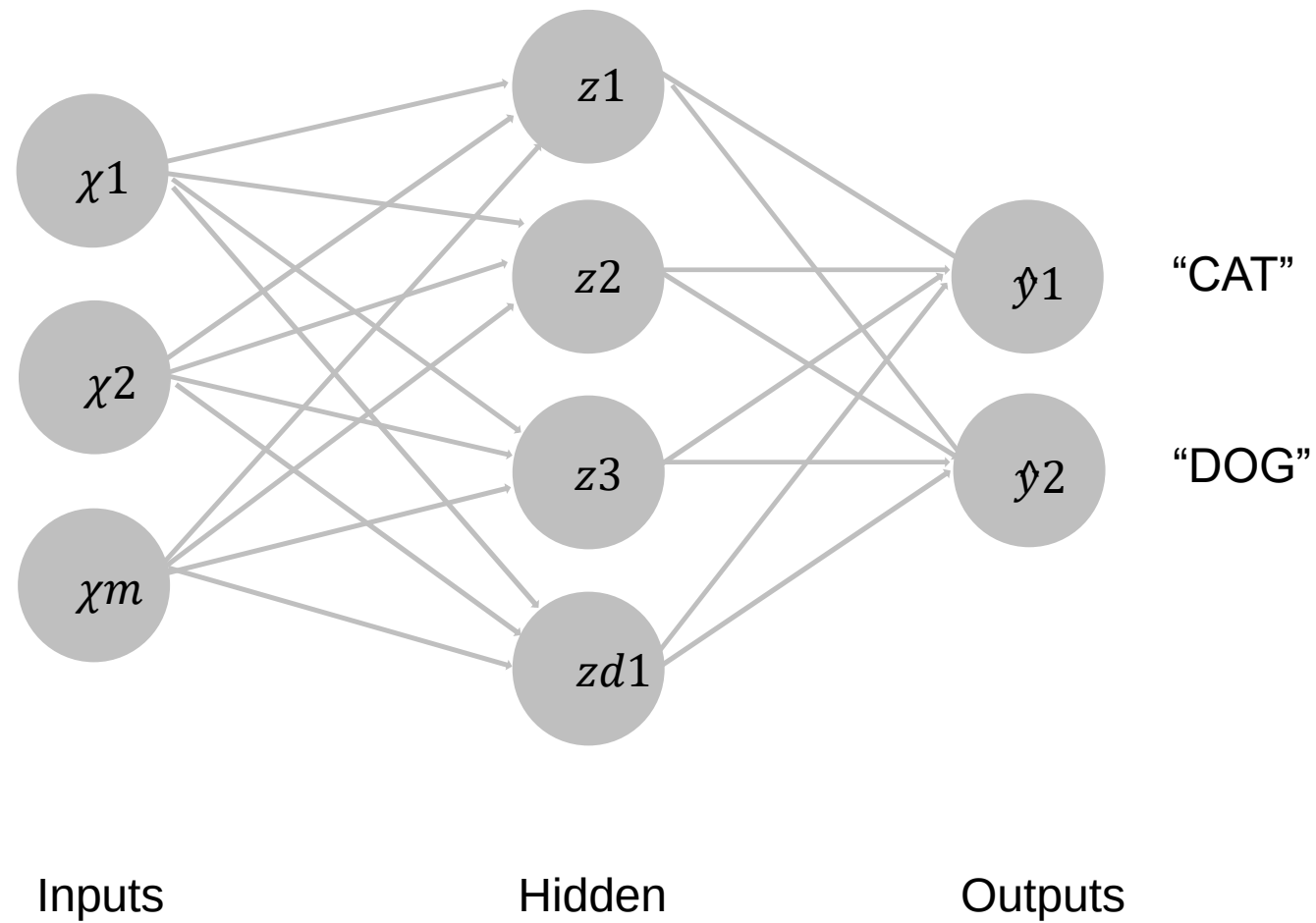


Building the network



CAT

$$\begin{bmatrix} \chi^1 \\ \chi^2 \\ \chi^m \end{bmatrix}$$

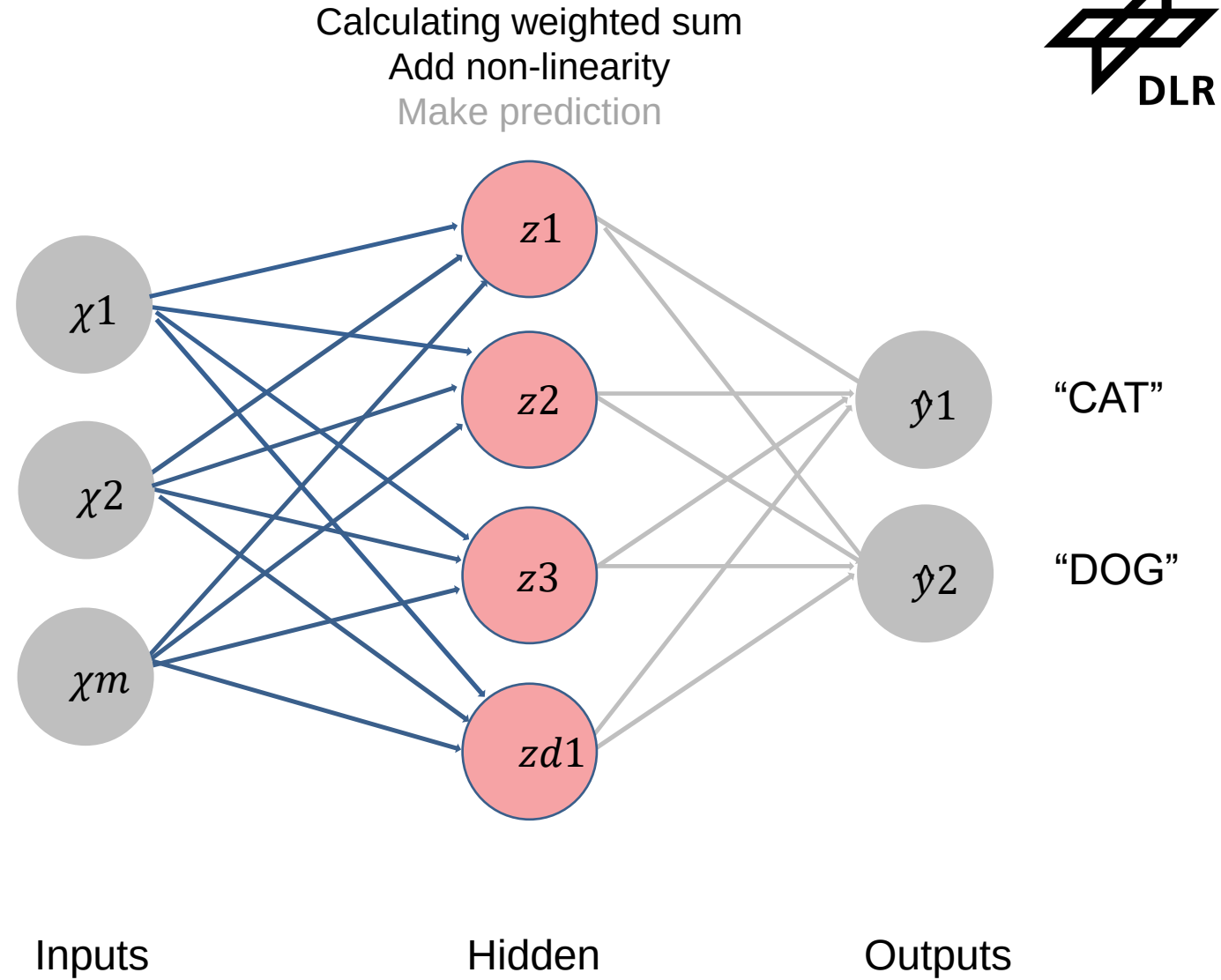


Forward Propagate Input



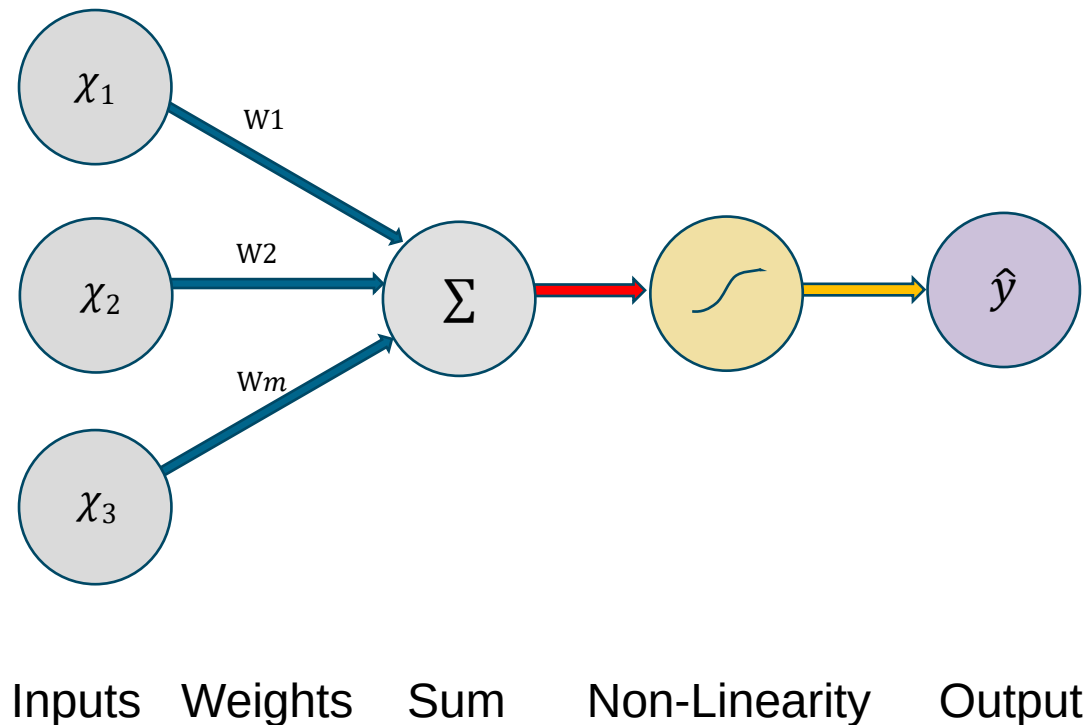
CAT

$$\begin{bmatrix} \chi^1 \\ \chi^2 \\ \chi^m \end{bmatrix}$$



Forward Propagate Input

A single neuron (perceptron)



Output

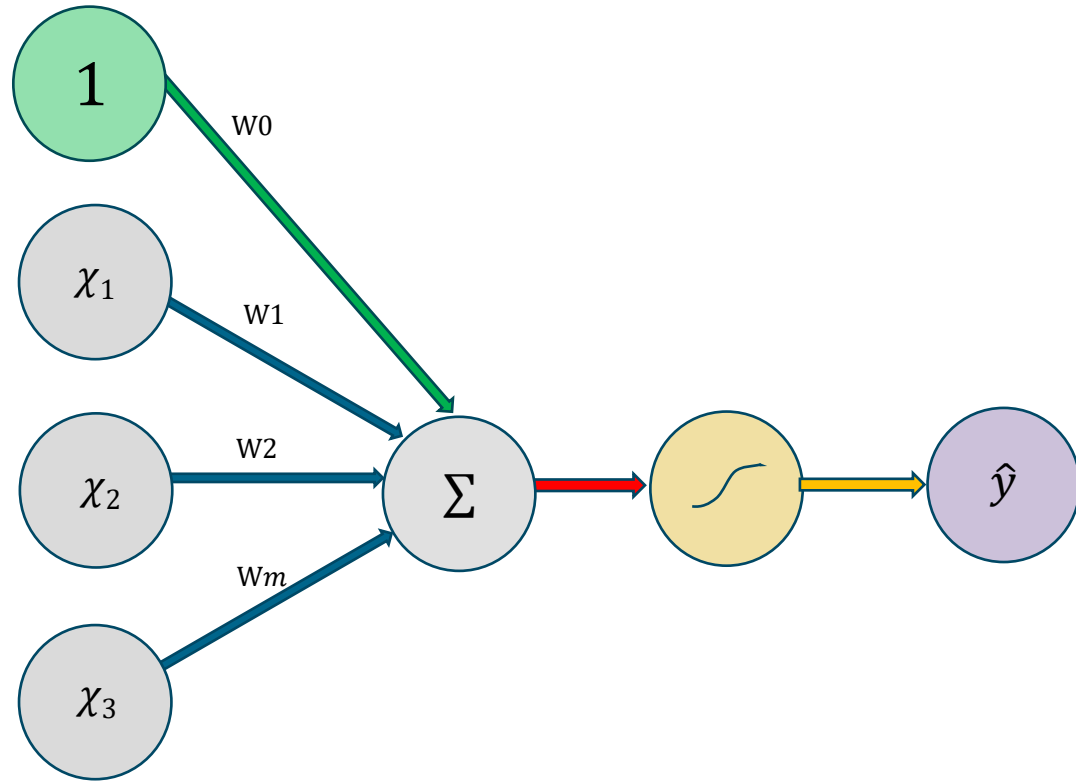
Linear combination of inputs

Non-linear activation function

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Forward Propagate Input

A single neuron (perceptron)



Inputs Weights Sum Non-Linearity Output

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

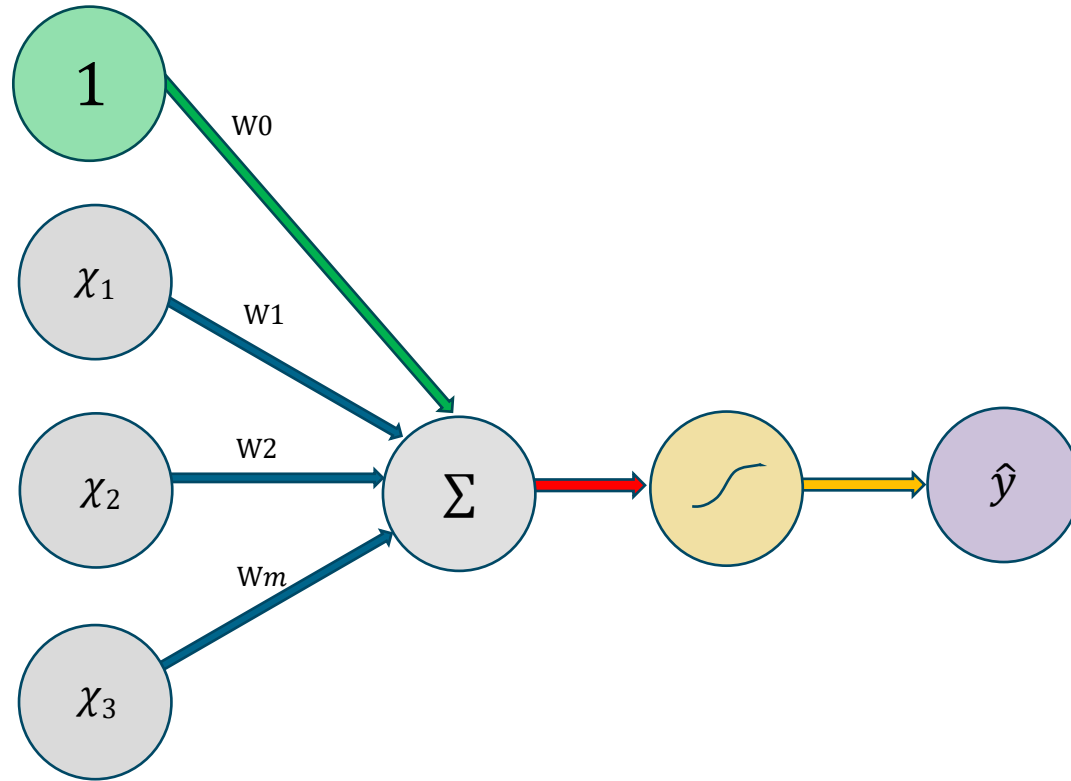
Bias

Forward Propagate Input

A single neuron (perceptron)



$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$



Inputs Weights Sum Non-Linearity Output

Vector/ Matrix
operations

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

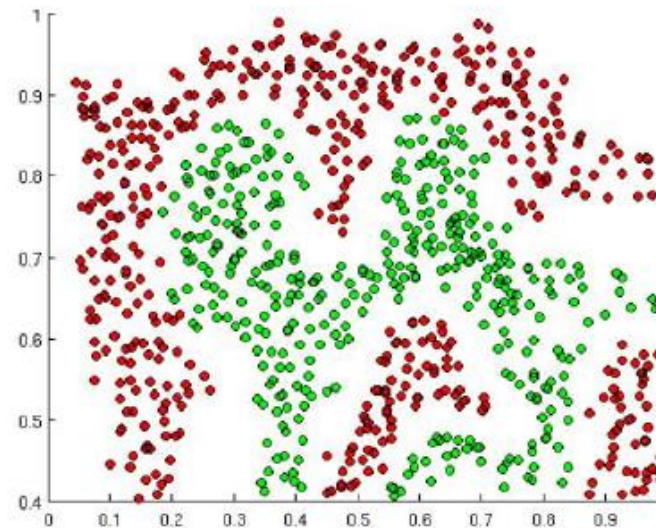
$$\mathbf{X} = \begin{bmatrix} X_1 \\ \vdots \\ X_m \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} W_1 \\ \vdots \\ W_m \end{bmatrix}$$

Forward Propagate Input

Activation functions

*The purpose of activation functions is to **introduce non-linearities** into the network*

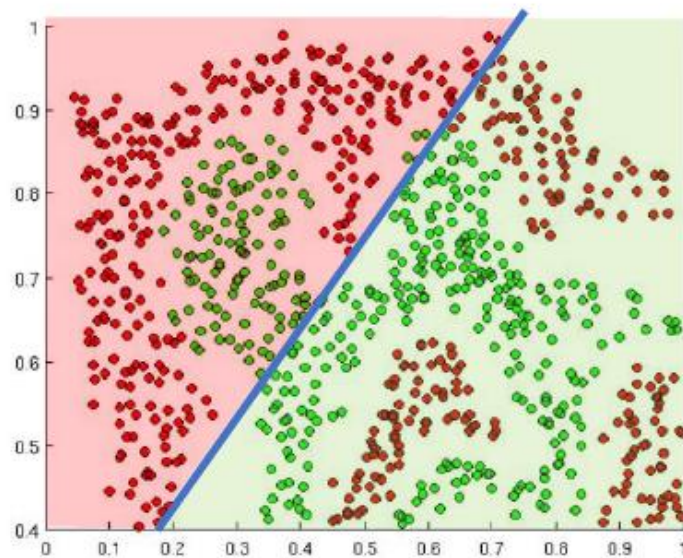


What if we wanted to build a Neural Network to distinguish green vs red points?

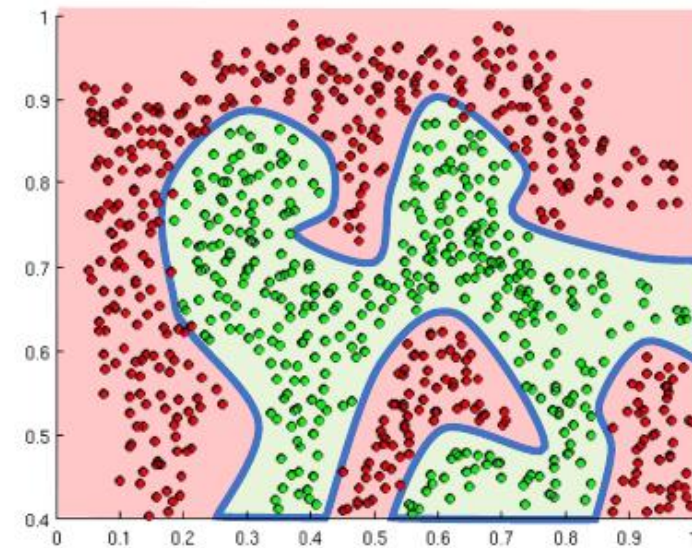
Forward Propagate Input

Activation functions

*The purpose of activation functions is to **introduce non-linearities** into the network*



Linear Activation functions produce linear decisions no matter the network size



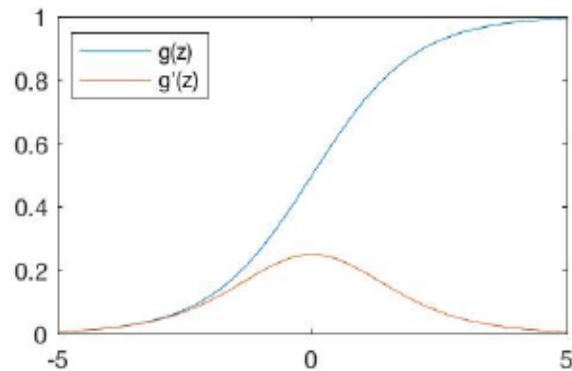
Non-linearities allow us to approximate arbitrarily complex functions

Forward Propagate Input

Non-linear Activation functions



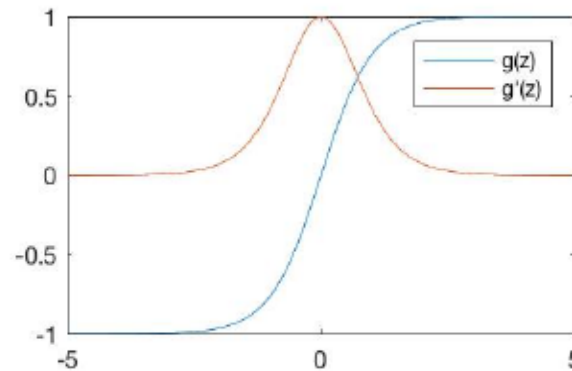
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

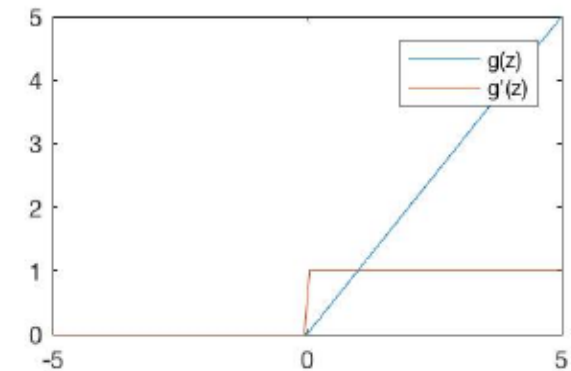
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)

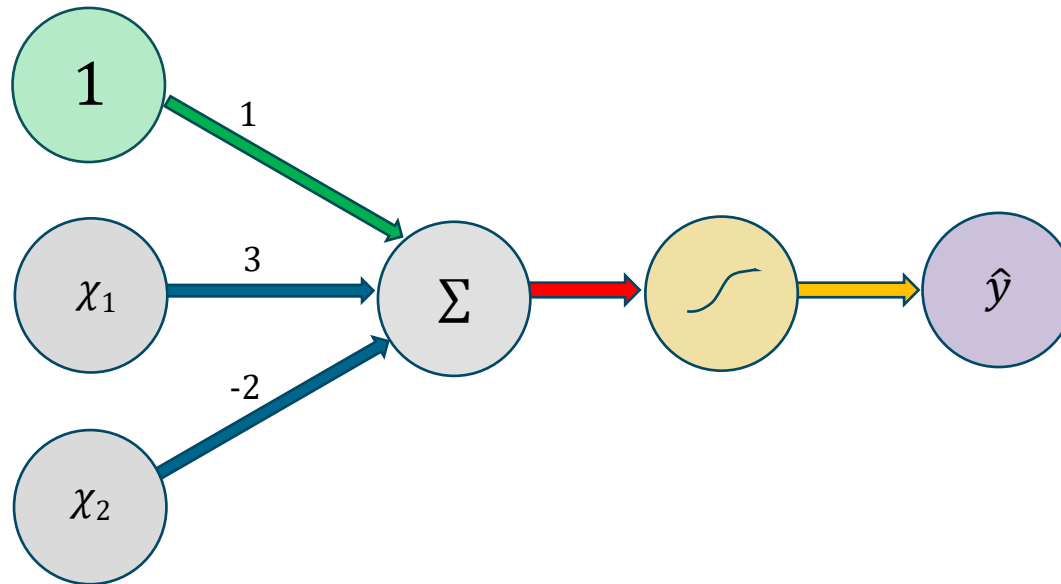


$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Forward Propagate Input

A single neuron (perceptron) - Example



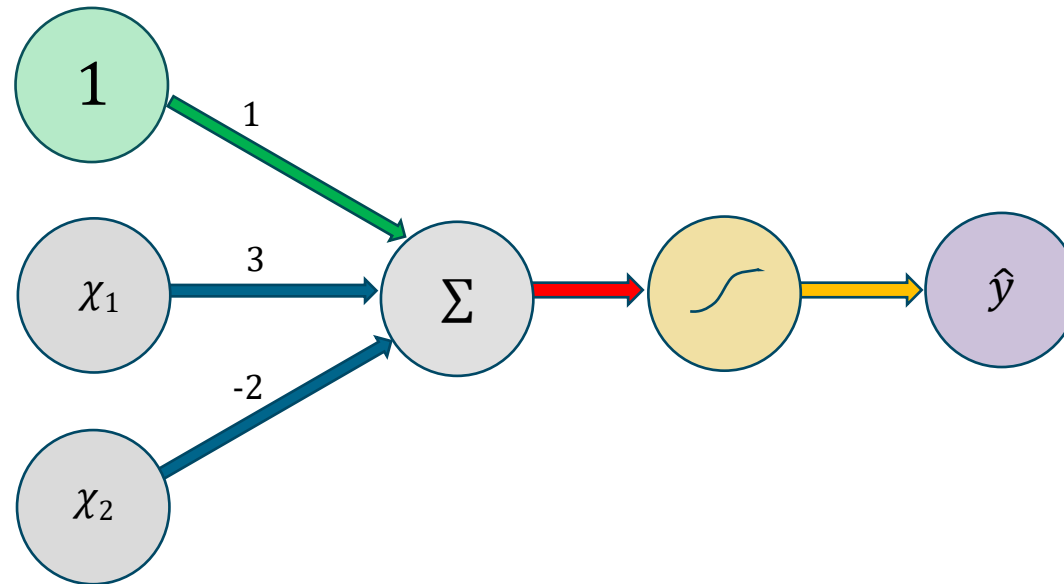
$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(\underbrace{1 + 3x_1 - 2x_2})\end{aligned} \quad \left| \quad \begin{aligned} w_0 &= 1 \\ \mathbf{W} &= \begin{bmatrix} 3 \\ -2 \end{bmatrix} \end{aligned}\right.$$

This is just a line in 2D

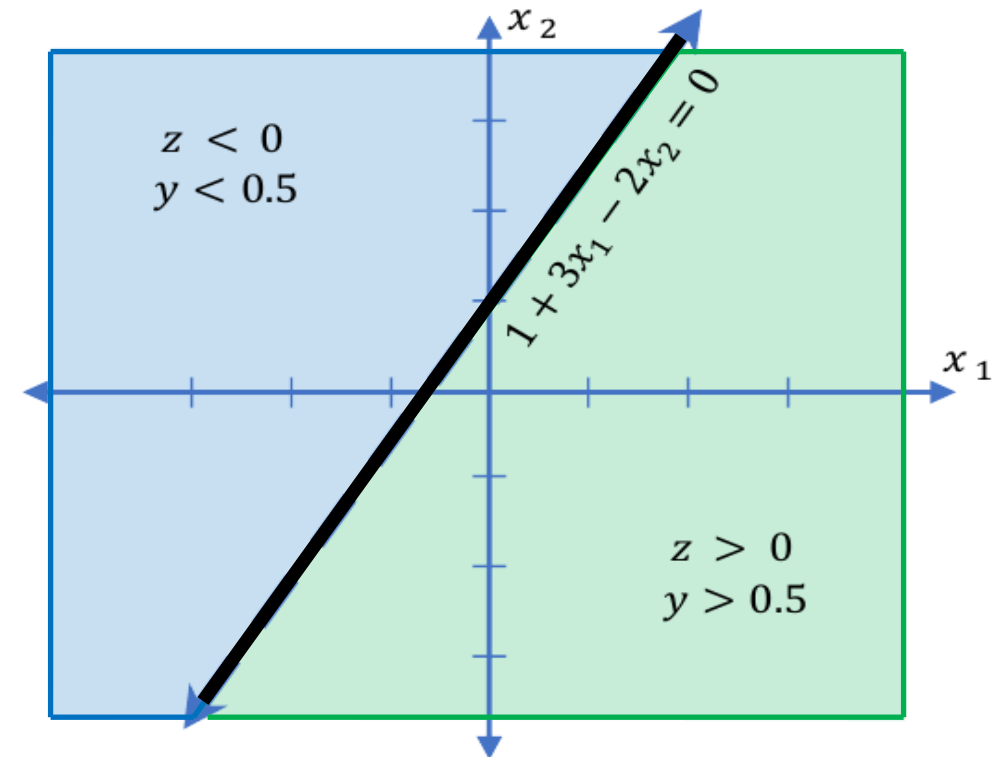
Forward Propagate Input

A single neuron (perceptron) - Example

$$w_0 = 1$$
$$W = \begin{bmatrix} 3 \\ -2 \end{bmatrix} \text{ DLR}$$



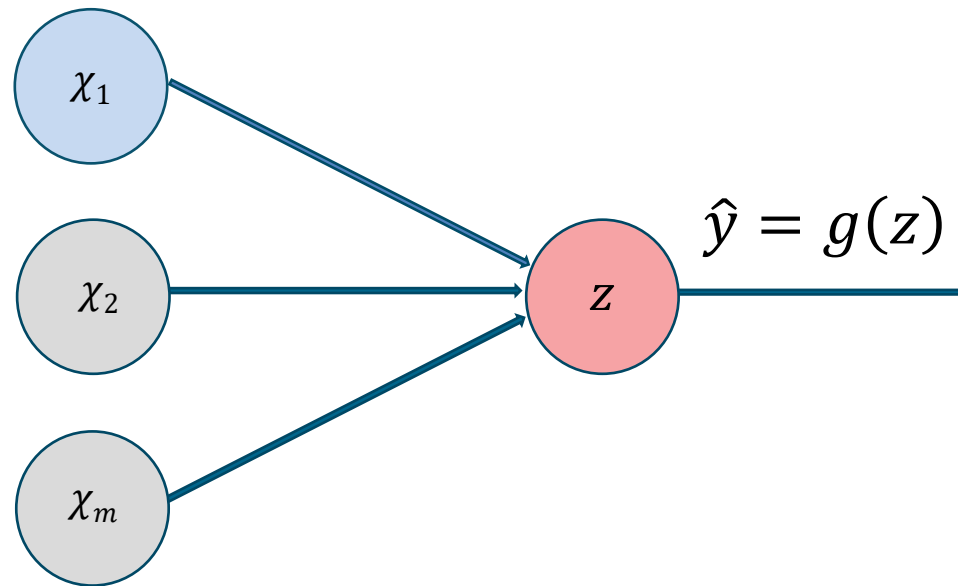
$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



Forward Propagate Input

Simplified neuron

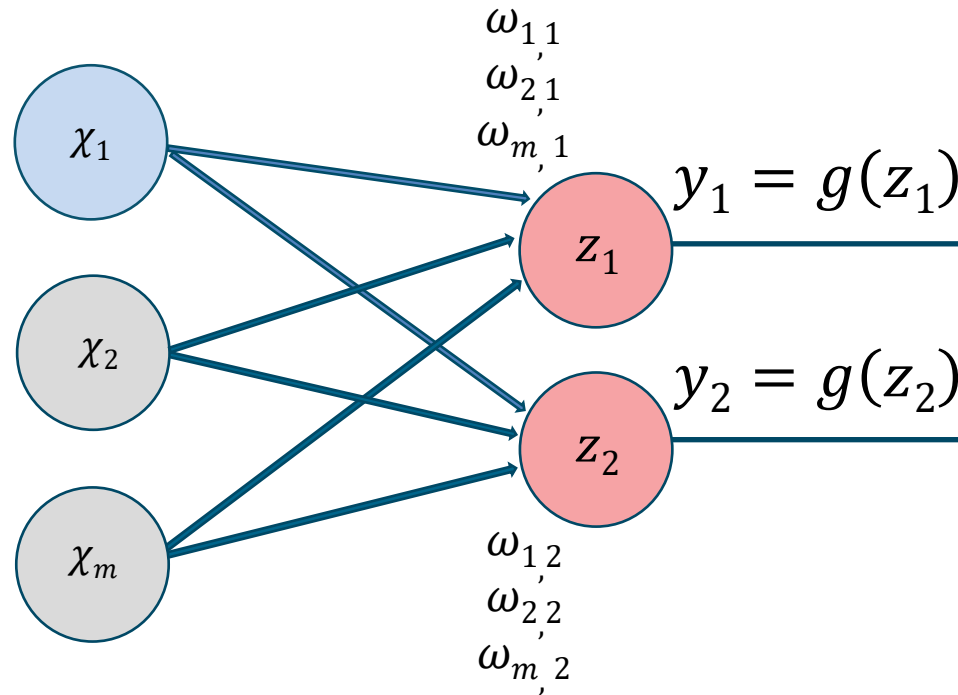
$$z = \left(w_0 + \sum_{j=1}^m x_j w_j \right)$$
The DLR logo, consisting of a stylized geometric design with the letters 'DLR' below it.



Forward Propagate Input

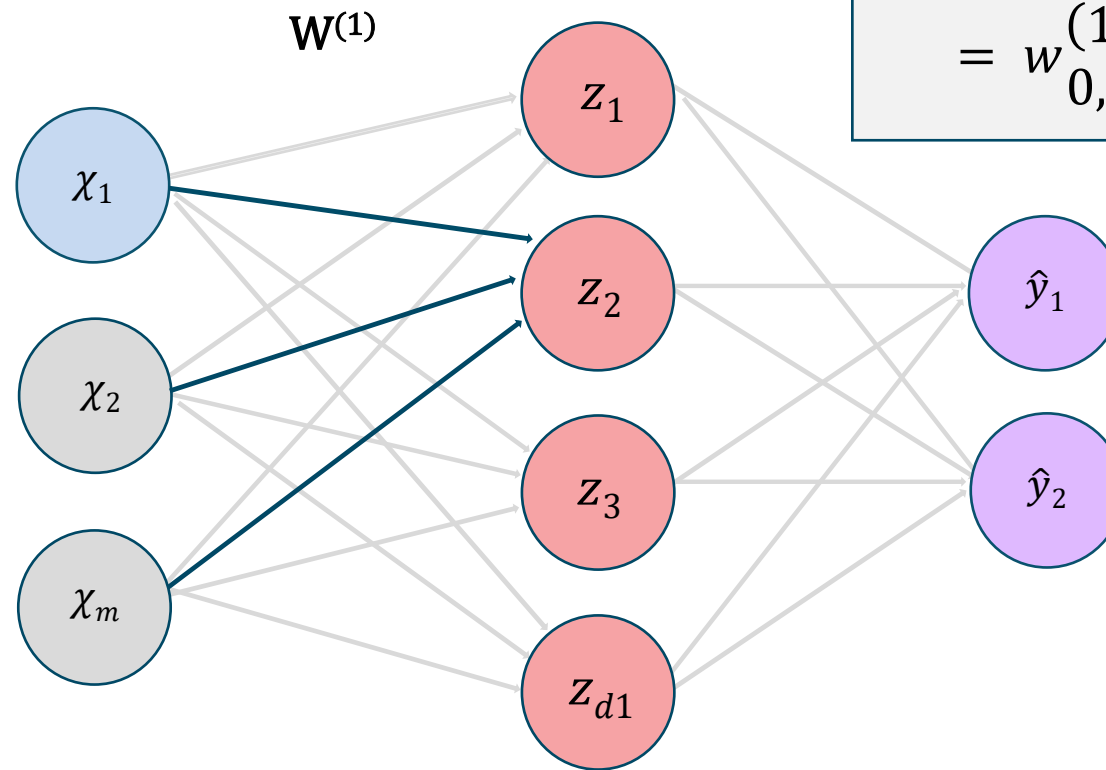
Multi-output neuron

$$z_i = \left(w_{0,i} + \sum_{j=1}^m x_j w_{j,i} \right)$$

Forward Propagate Input

A single-layer neural network



$$z_2 = \left(w_{0,2}^{(1)} + \sum_{j=1}^m x_j \left(w_{j,2}^{(1)} \right) \right)$$
$$= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)}$$

A red arrow points down to the summation term in the first equation, and a red bracket with an upward arrow points to the expanded summation in the second equation.

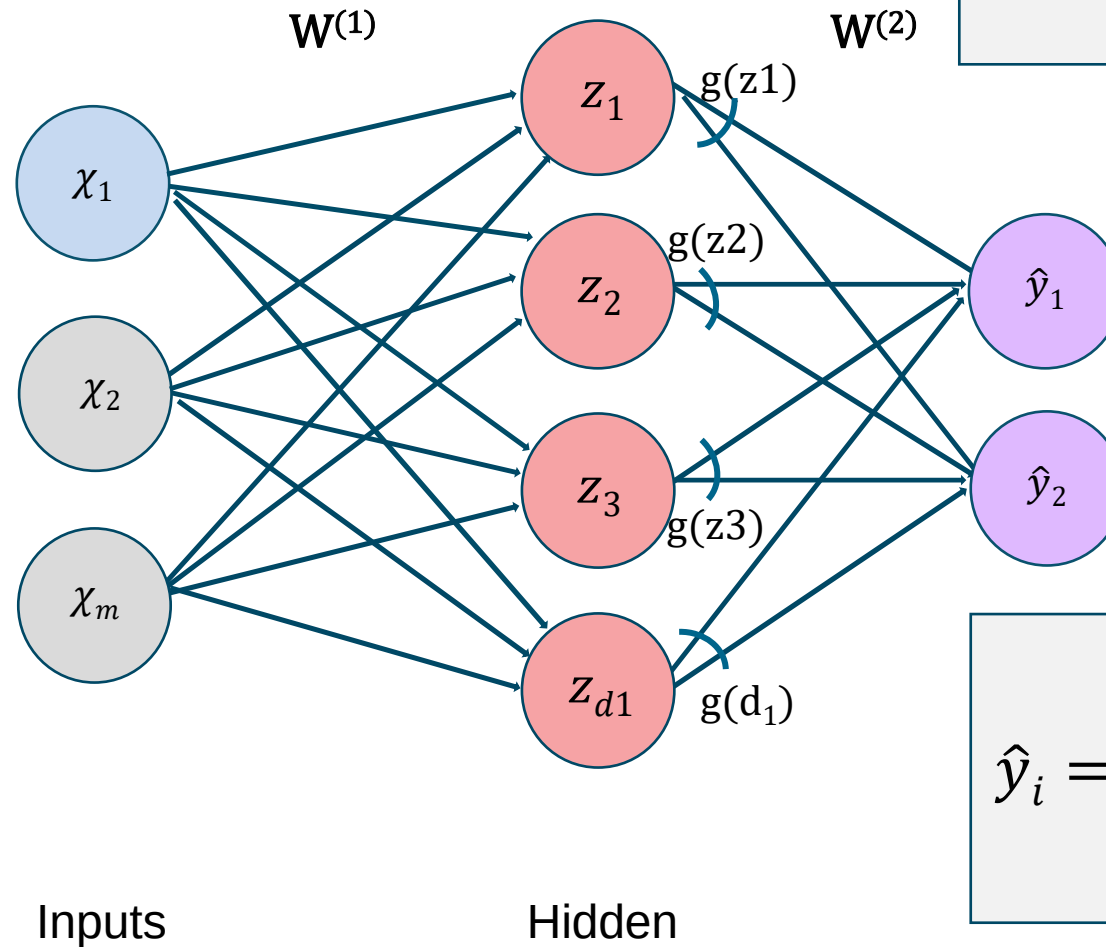
Inputs

Hidden

Final Output

Forward Propagate Input

A single-layer neural network

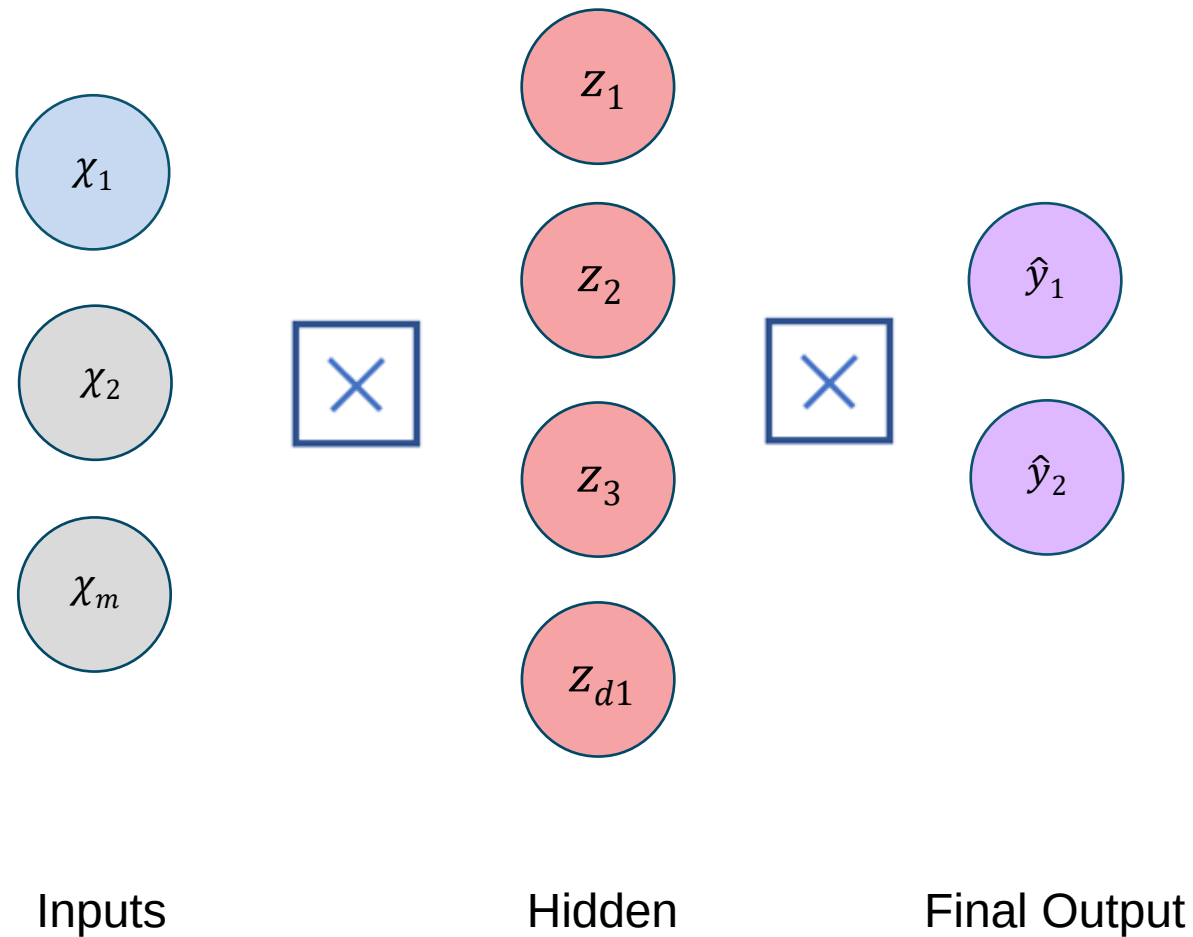


$$z_i = \left(w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \right)$$

$$\hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)} \right)$$

Forward Propagate Input

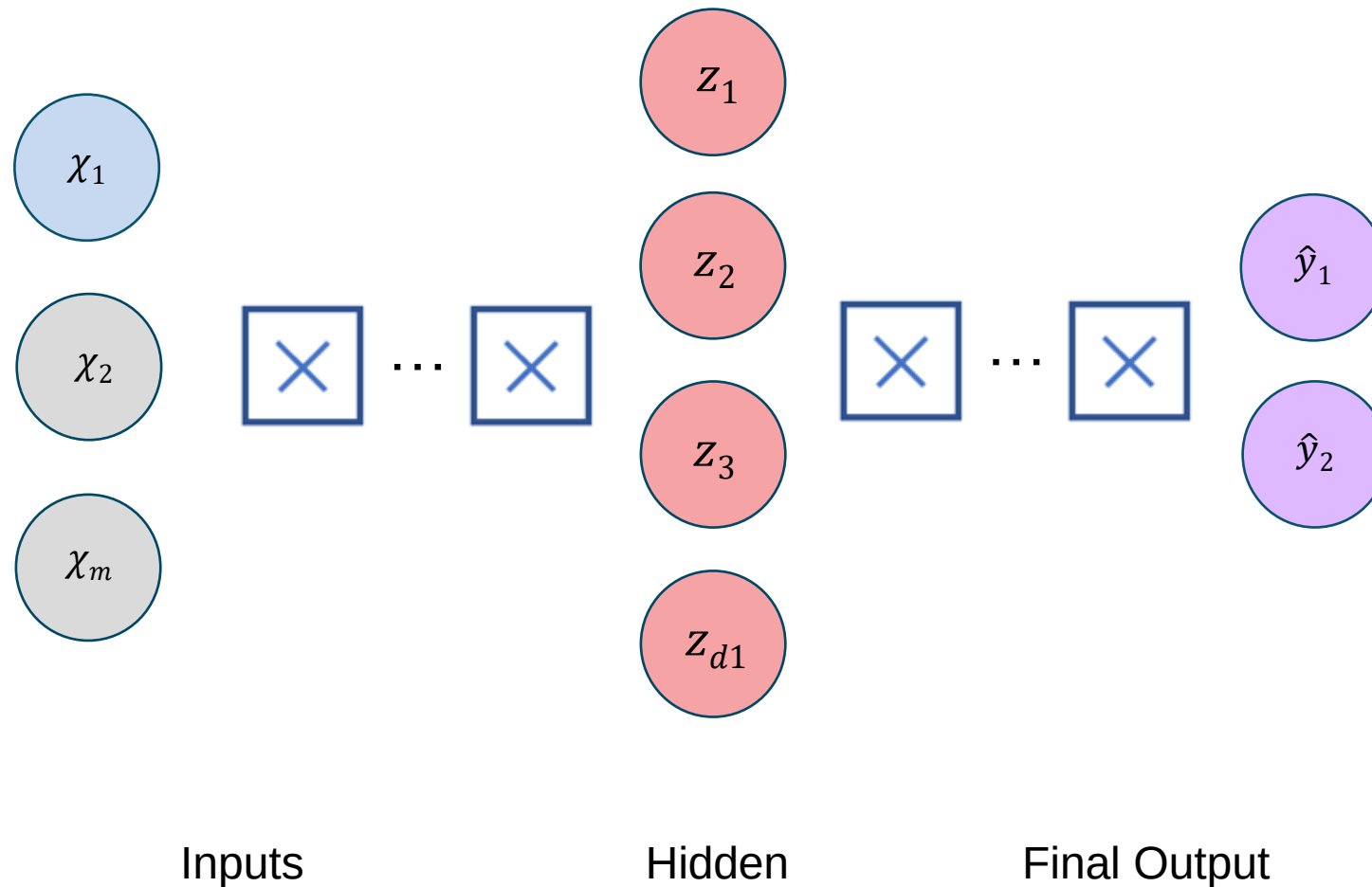
A single-layer neural network



Forward Propagate Input

Deep neural network

$$Z_{k,i} = \left(w_{0,i}^{(k)} + \sum_{j=1}^{d_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)} \right)$$

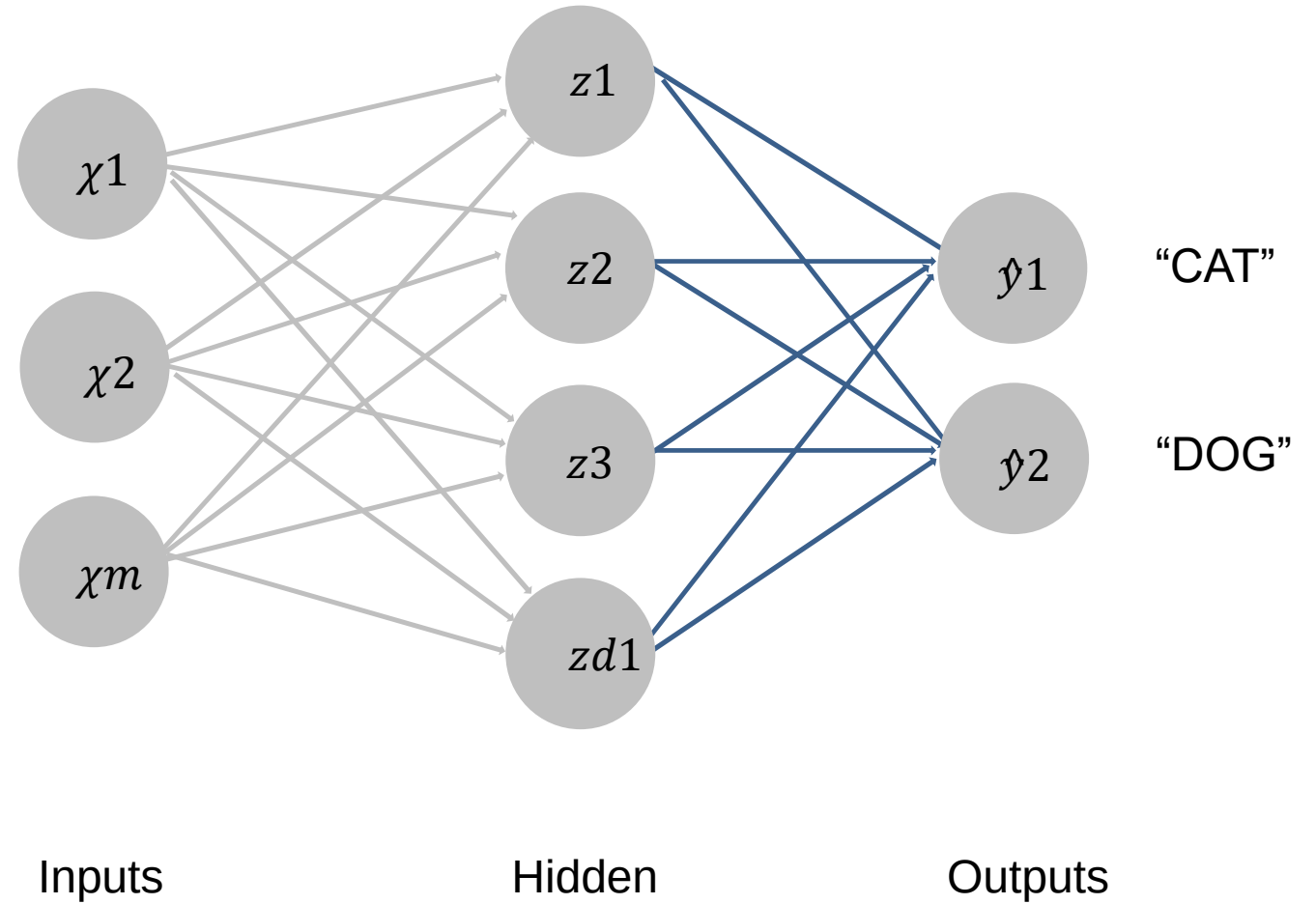


Forward Propagate Input



CAT

$$\begin{bmatrix} \chi^1 \\ \chi^2 \\ \chi^m \end{bmatrix}$$



Forward Propagate Input

Softmax activation

(= softargmax = normalized exponential function)

$$S(f_{y_i}) = \frac{e^{f_{y_i}}}{\sum_j e^{f_j}}$$

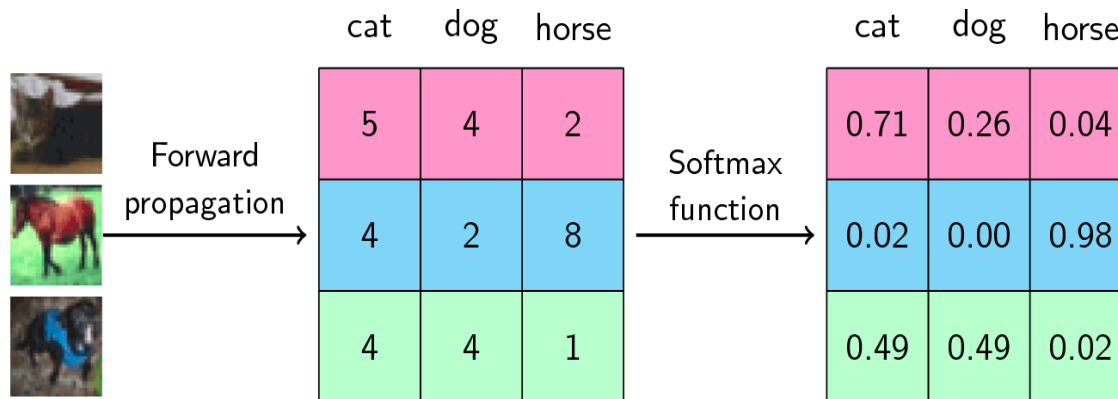
$$S(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Input pixels, x

Feedforward output, y_i

Softmax output, $S(y_i)$



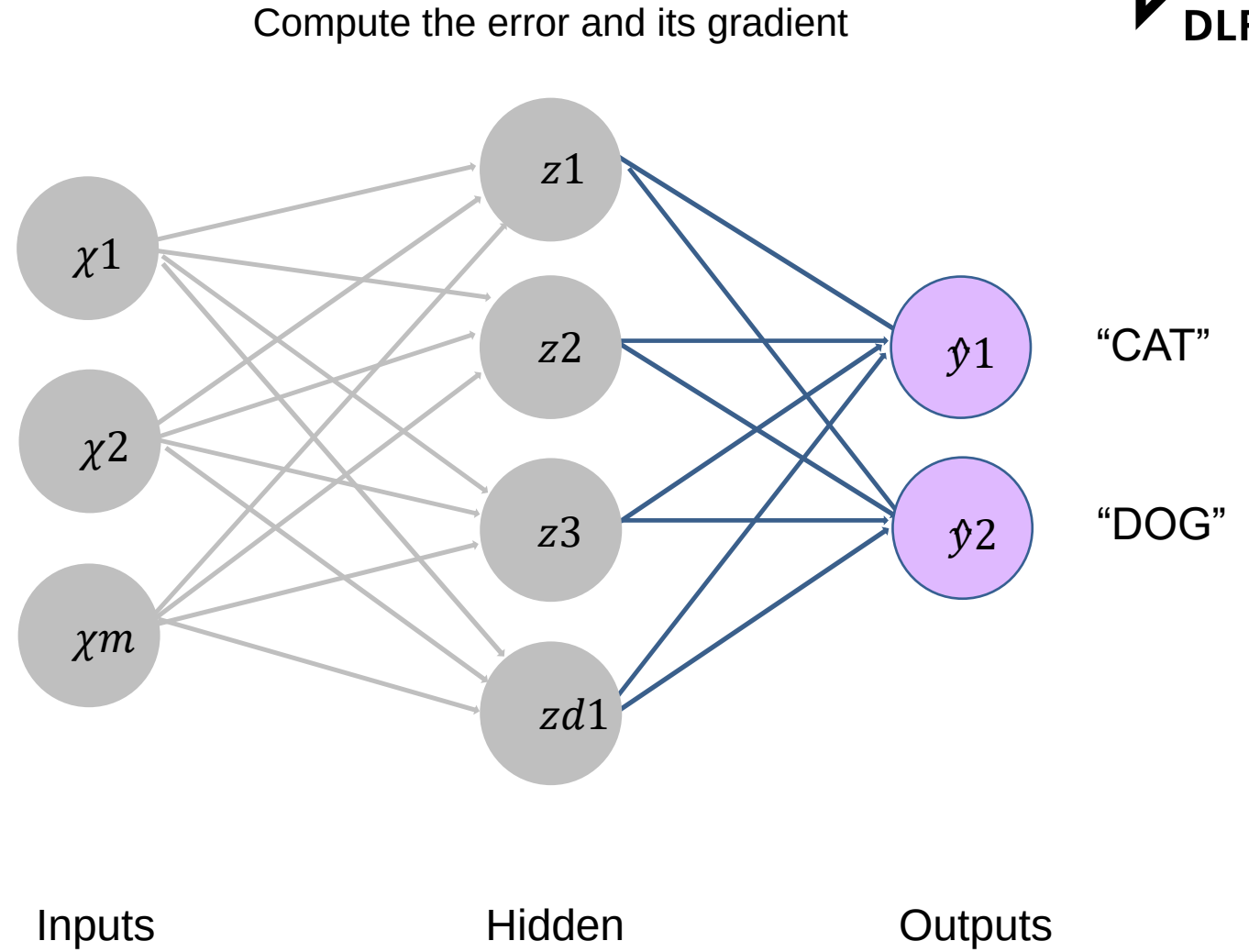
<https://ljvmiranda921.github.io/notebook/2017/08/13/softmax-and-the-negative-log-likelihood/>

Evaluate Prediction



CAT

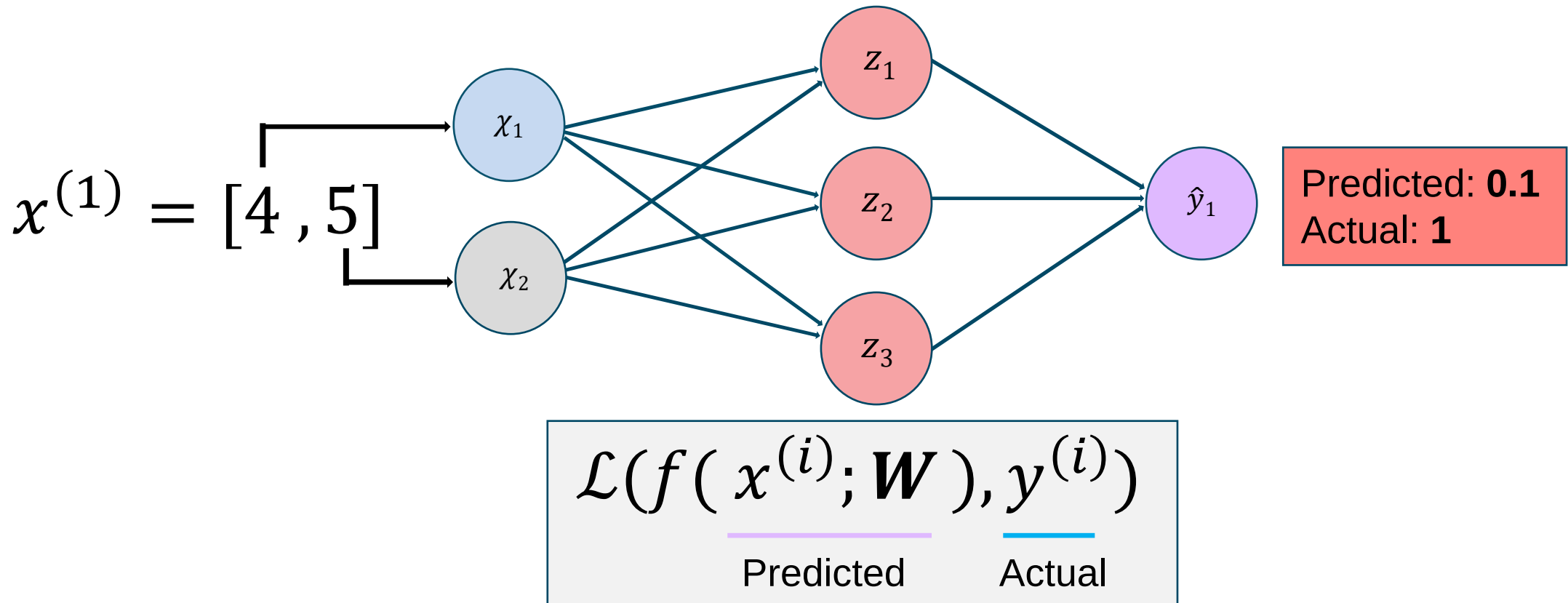
$$\begin{bmatrix} \chi^1 \\ \chi^2 \\ \chi^m \end{bmatrix}$$



Evaluate Prediction

Loss function

The **loss** of our network measures the cost incurred from incorrect predictions

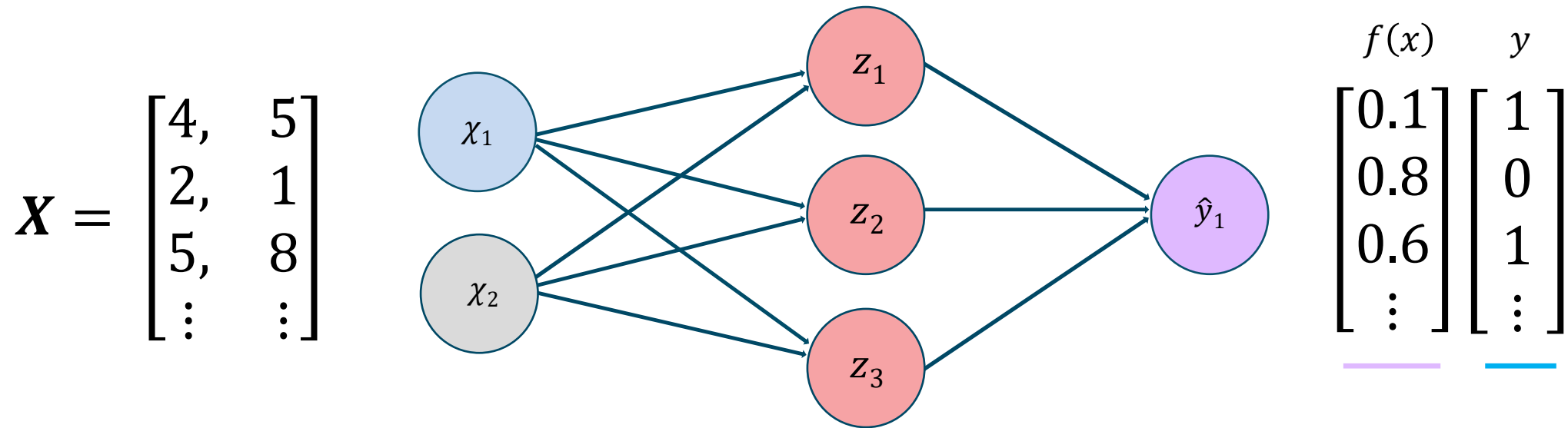


Evaluate Prediction

Loss function



The **empirical loss** measures the total loss over our entire dataset



Also known as:

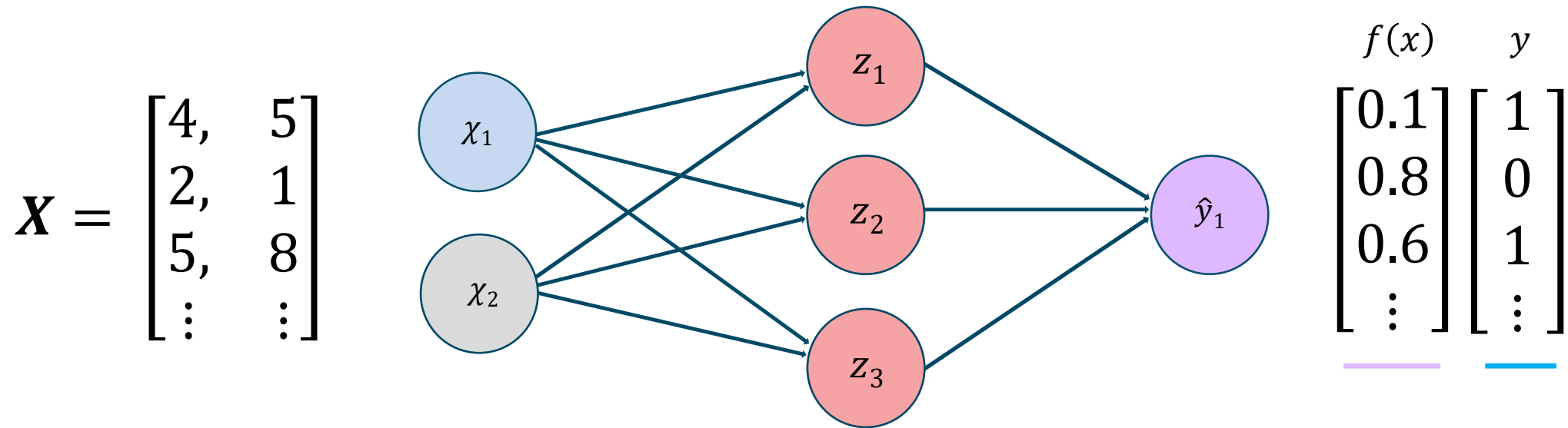
- Objective function
- Cost function
- Empirical Risk

$$J(W) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Evaluate Prediction

Binary cross-entropy loss

The **Cross entropy loss** can be used with models that output a probability between 0 and 1



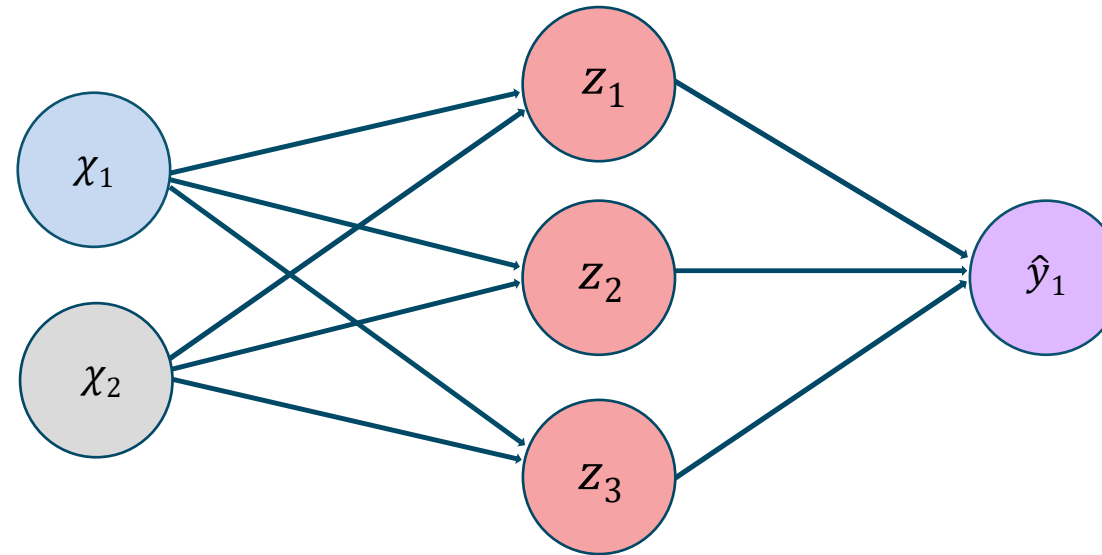
$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log(1 - \underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}})$$

Evaluate Prediction

Mean squared error loss

Mean squared error loss can be used with regression models that output continuous real numbers

$$X = \begin{bmatrix} 4, & 5 \\ 2, & 1 \\ 5, & 8 \\ \vdots & \vdots \end{bmatrix}$$



$$\begin{array}{cc} f(x) & y \\ \begin{bmatrix} 30 \\ 80 \\ 85 \\ \vdots \end{bmatrix} & \begin{bmatrix} 90 \\ 20 \\ 95 \\ \vdots \end{bmatrix} \end{array}$$

Final Grades (percentage)

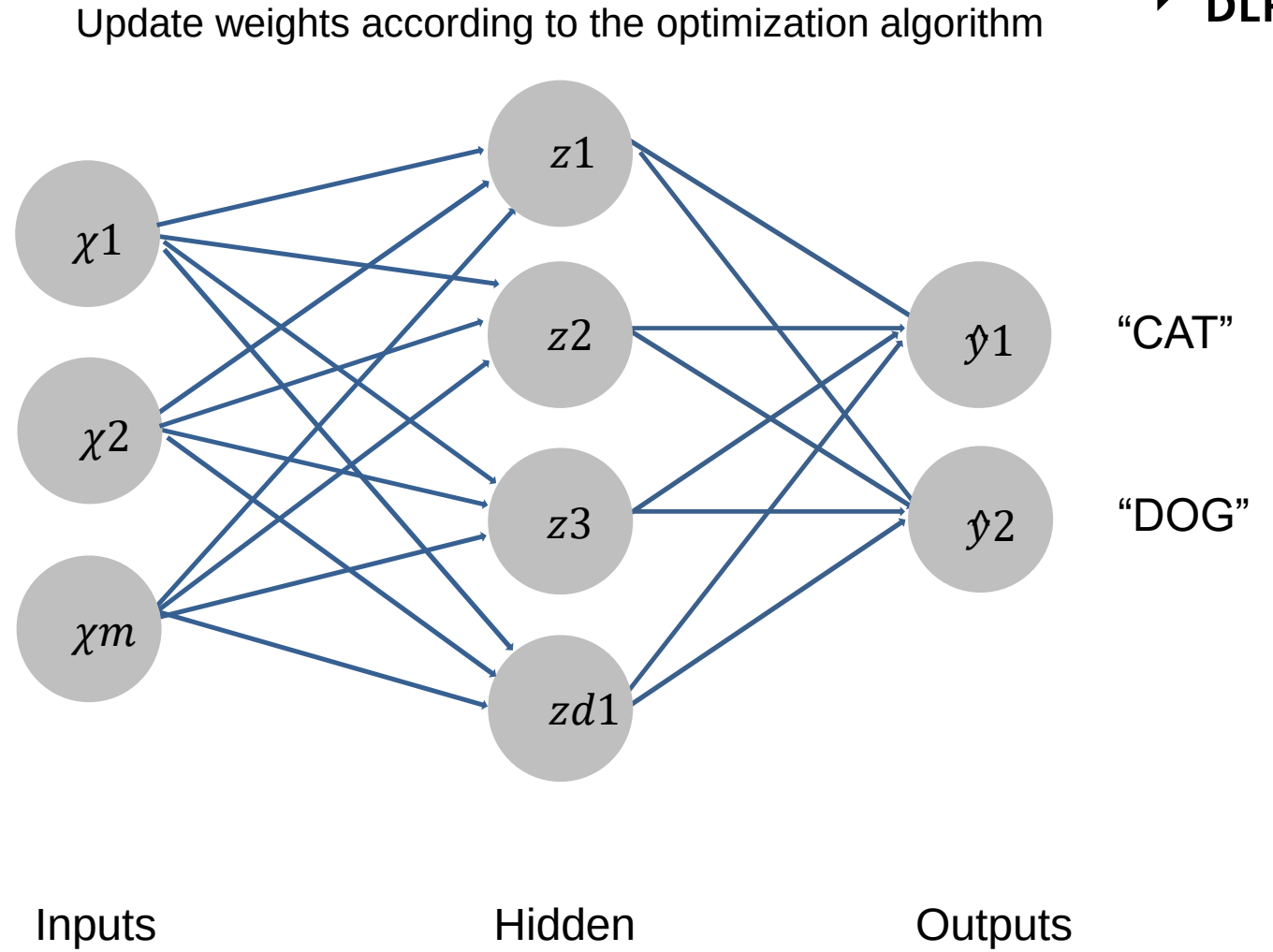
$$J(W) = \frac{1}{n} \sum_{i=1}^n (\underbrace{y^{(i)}}_{\text{Actual}} - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}})^2$$

Back propagate



CAT

$$\begin{bmatrix} \chi^1 \\ \chi^2 \\ \chi^m \end{bmatrix}$$



Back Propagate

Loss optimization



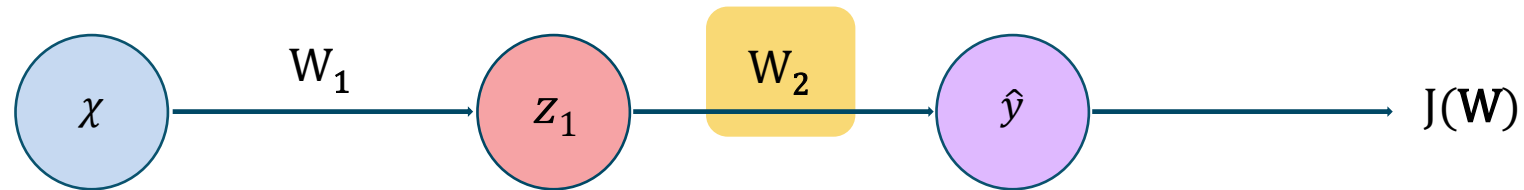
For **optimization** we want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$w_0 = 1$

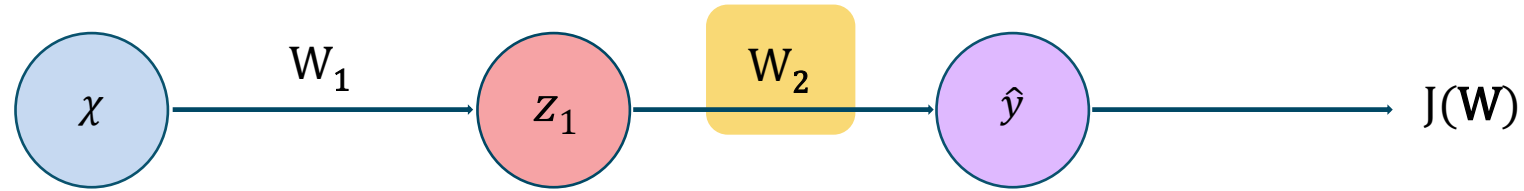
$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} J(\mathbf{W}) \quad \left| \quad \mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$$

Back Propagate



How does a small change in one weight (ex. w_2) affect the final loss $J(W)$

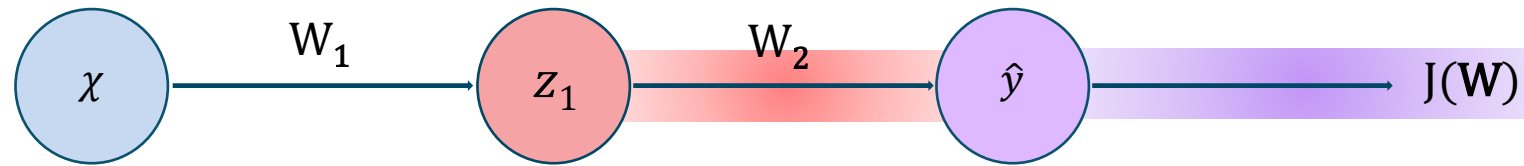
Back Propagate



$$\frac{\partial J(W)}{\partial w_2} =$$

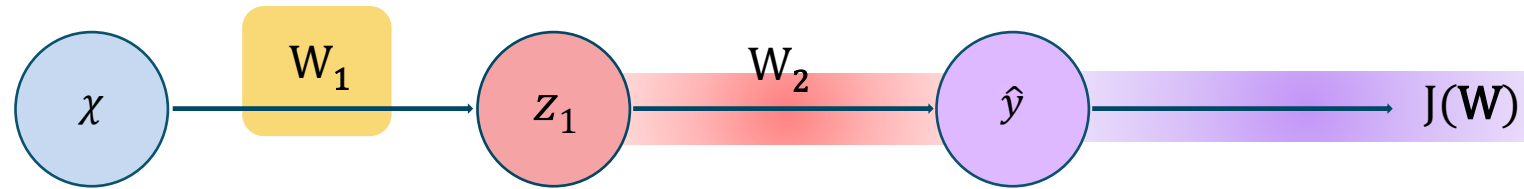
Let's use the chain rule!

Back Propagate



$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

Back Propagate

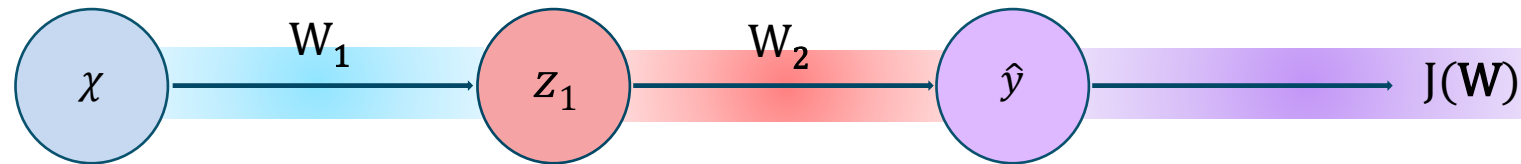


$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

↑
Apply chain rule!

↑
Apply chain rule!

Back Propagate



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

(The terms in the equation are color-coded to match the diagram: $\frac{\partial J(W)}{\partial \hat{y}}$ is purple, $\frac{\partial \hat{y}}{\partial z_1}$ is red, and $\frac{\partial z_1}{\partial w_1}$ is light blue.)

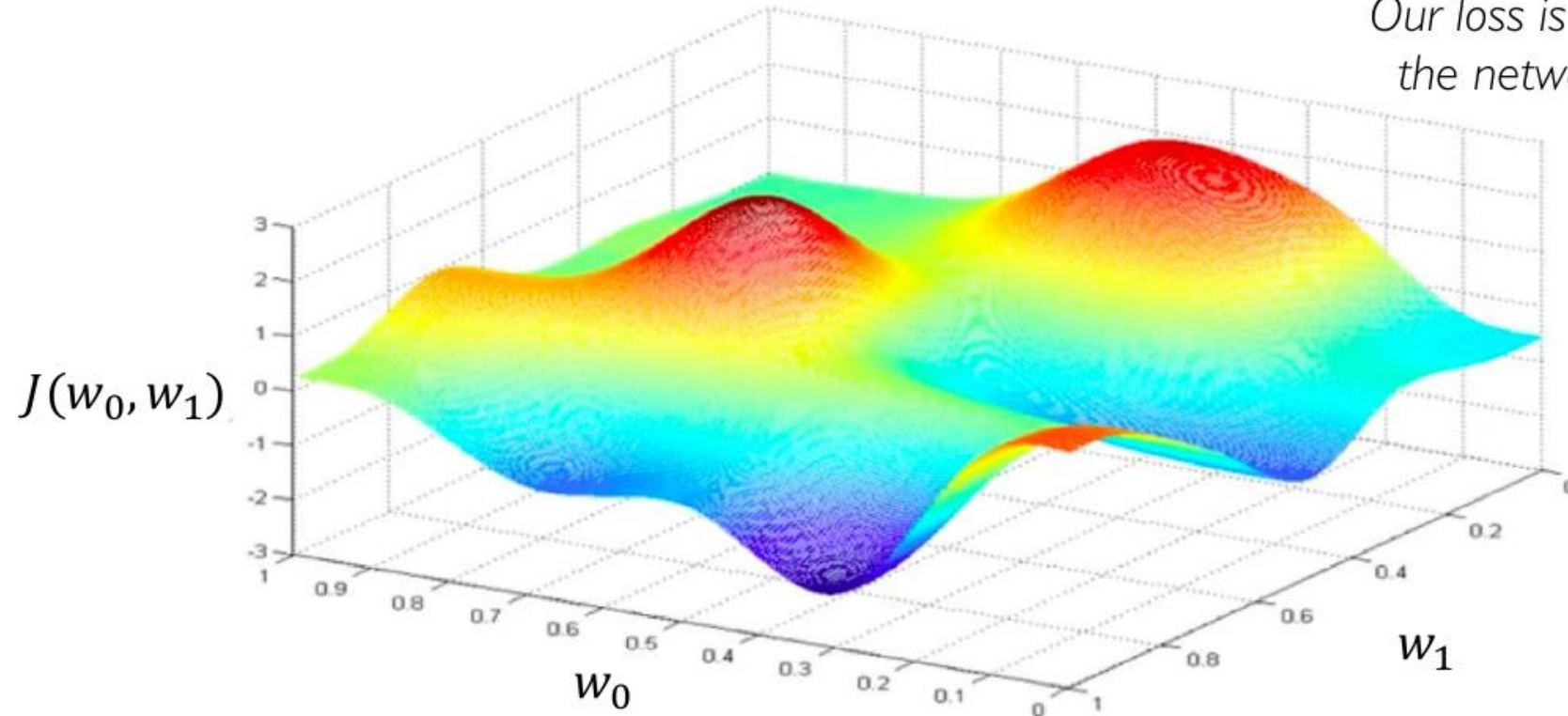
Repeat this for **every weight in the network** using gradients from layers

Back Propagate

Loss optimization

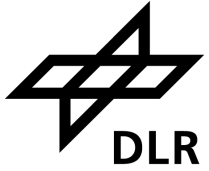
$$W^* = \underset{W}{\operatorname{argmin}} J(W)$$

Remember:
*Our loss is a function of
the network weights!*

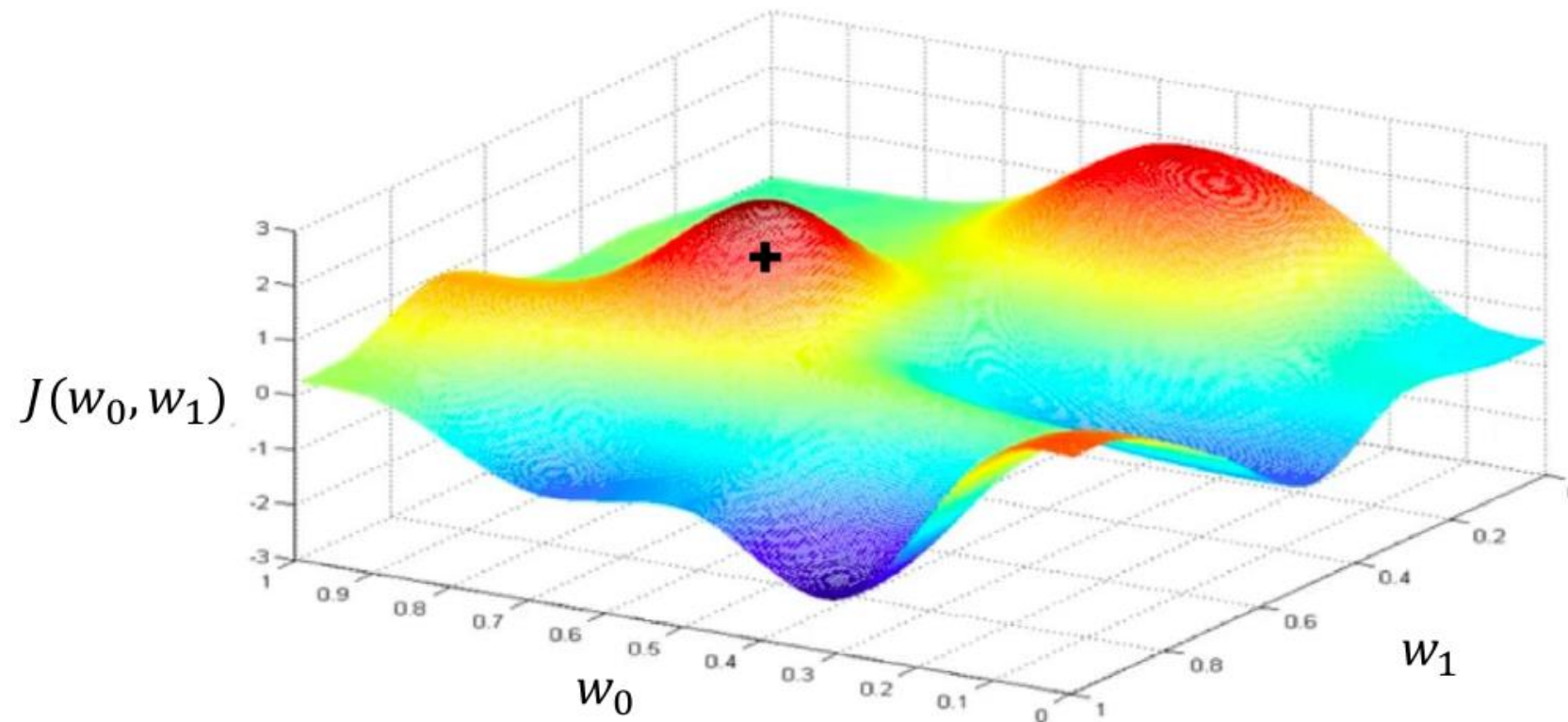


Back Propagate

Gradient descent

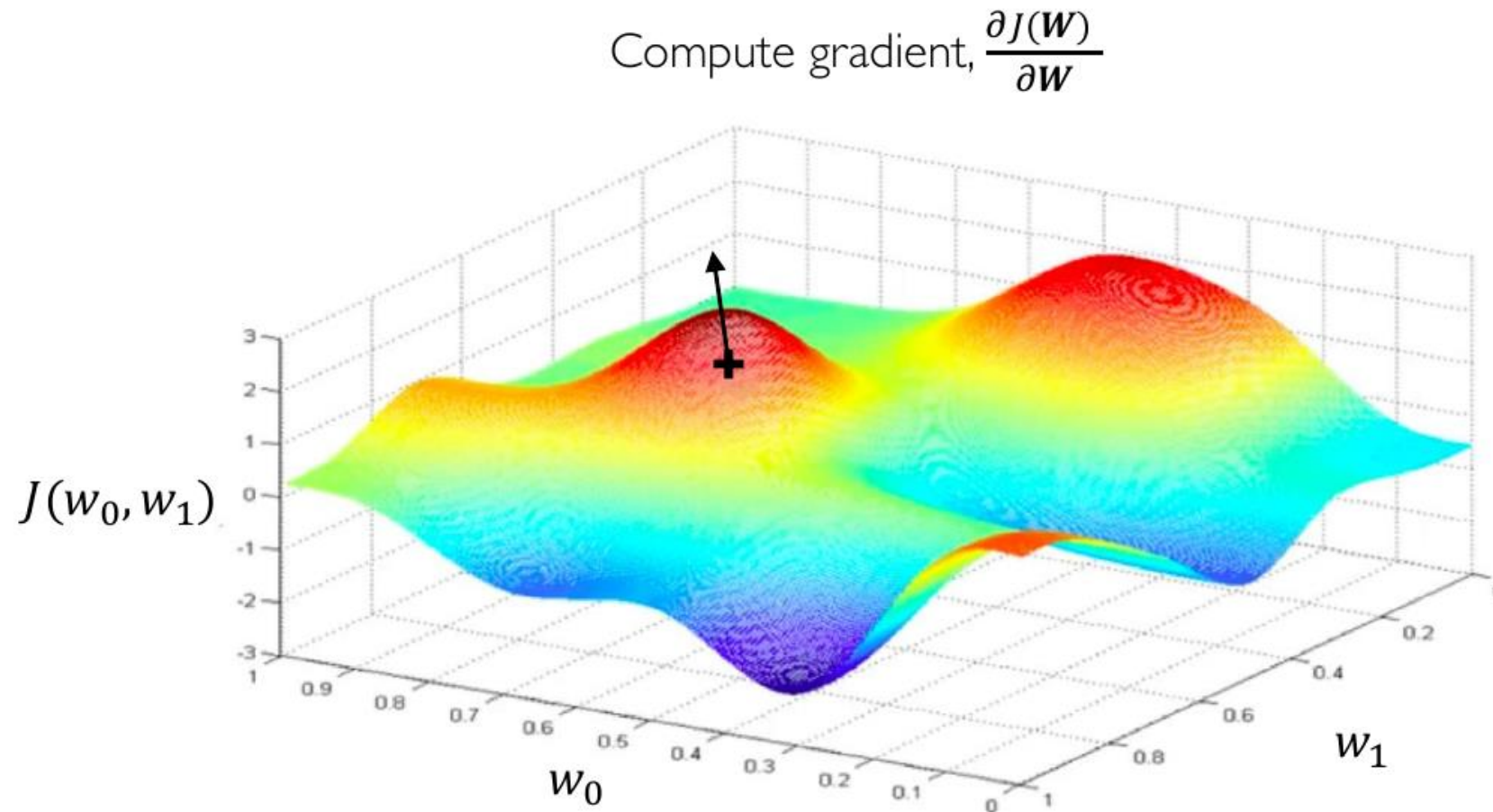


Randomly pick an initial (w_0, w_1)



Back Propagate

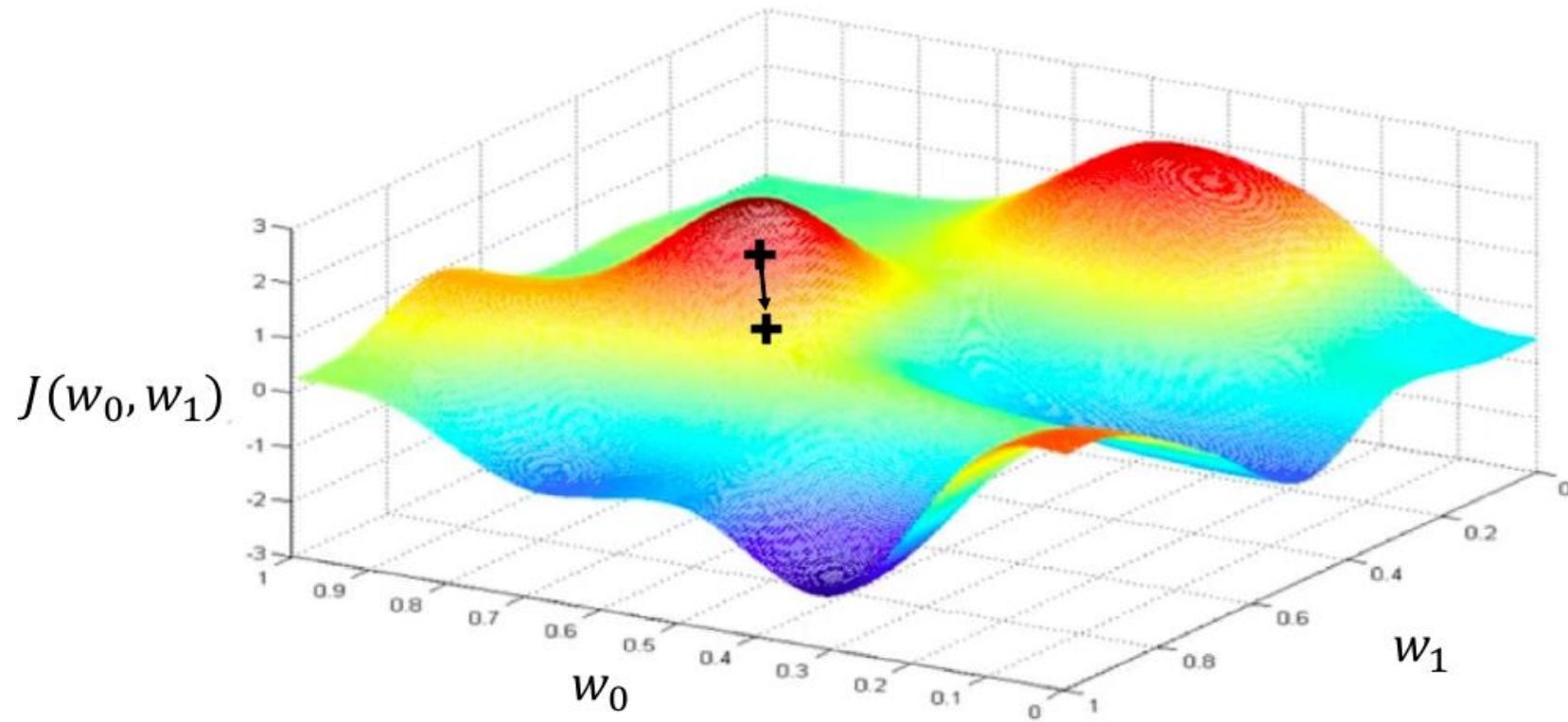
Gradient descent



Back Propagate

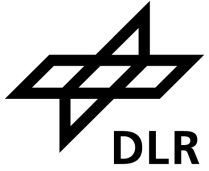
Gradient descent

Take small step in opposite direction of gradient

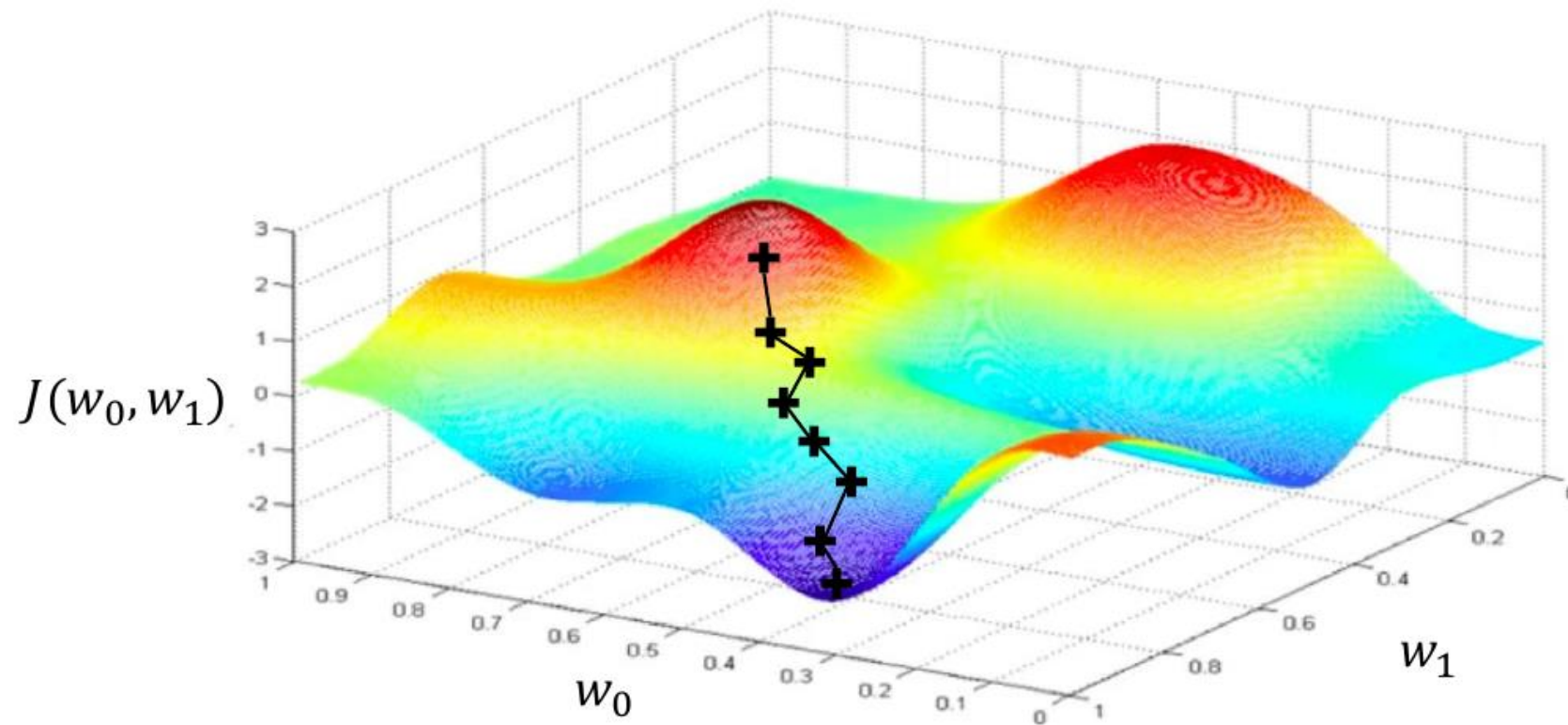


Back Propagate

Gradient descent



Repeat until convergence



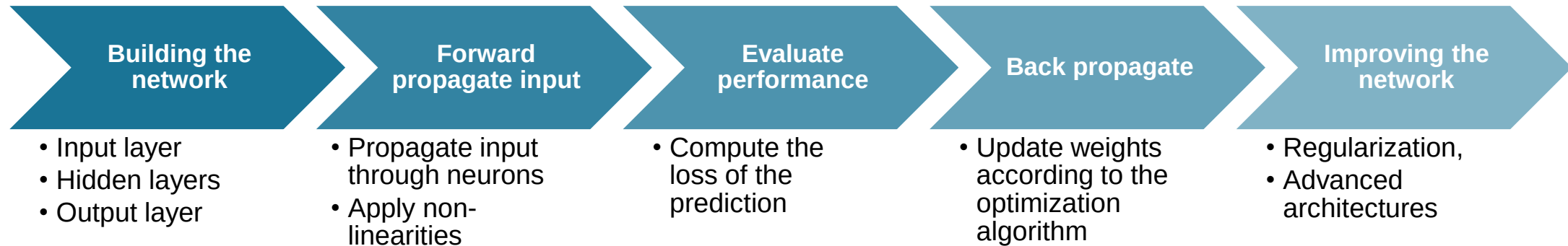
Back Propagate

Gradient descent



Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights



Next:

- Hands-on
- Theory Part II:
 - Advanced concept: Regularization
 - Convolutional Neural Networks

Topic: **Introduction to Deep Learning**
Part I – Introduction and Basics

Date: 2026-05-28

Author: Auliya Fitri, Sheetal Sahoo, Sreerag Naveenachandran

Institute: Data Science

Image sources: All images “DLR (CC BY-NC-ND 3.0)” unless otherwise stated